



Inferencing on Edge Devices: A Time- and Space-aware Co-scheduling Approach

DANNY PEREIRA, Indian Institute of Technology, Kharagpur, India

ANIRBAN GHOSE, Indian Institute of Technology, Kharagpur, India

SUMANA GHOSH, Indian Statistical Institute, Kolkata, India

SOUMYAJIT DEY, Indian Institute of Technology, Kharagpur, India

Neural Network (NN)-based real-time inferencing tasks are often co-scheduled on GPGPU-style edge platforms. Existing works advocate using different NN parameters for the same detection task in different environments. However, realizing such approaches remains challenging, given accelerator devices' limited on-chip memory capacity. As a solution, we propose a multi-pass, time- and space-aware scheduling infrastructure for embedded platforms with GPU accelerators. The framework manages the residency of NN parameters in the limited on-chip memory while simultaneously dispatching relevant compute operations. The mapping decisions for memory operations and compute operations to the underlying resources of the platform are first determined in an offline manner. For this, we proposed a constraint solver-assisted scheduler that optimizes for schedule makespan. This is followed by memory optimization passes, which take the memory budget into account and accordingly adjust the start times of memory and compute operations. Our approach reports a 74%–90% savings in peak memory utilization with 0%–33% deadline misses for schedules that suffer miss percentage in ranges of 25%–100% when run using existing methods.

CCS Concepts: • **Computing methodologies** → **Optimization algorithms**; • **Computer systems organization** → **Embedded software**;

Additional Key Words and Phrases: Convolutional neural network, edge device, GPU, Satisfiability Modulo Theories

ACM Reference format:

Danny Pereira, Anirban Ghose, Sumana Ghosh, and Soumyajit Dey. 2023. Inferencing on Edge Devices: A Time- and Space-aware Co-scheduling Approach. *ACM Trans. Des. Autom. Electron. Syst.* 28, 3, Article 38 (March 2023), 33 pages.
<https://doi.org/10.1145/3576197>

1 INTRODUCTION

Efficient deployment of multiple **Neural Network (NN)**-based inference tasks on resource-constrained edge devices on time presents an important and challenging research problem. **Advanced Driver-Assistance Systems (ADAS)** are a widespread use case where multiple detection tasks periodically execute and contend for memory and compute resources of the underlying

Authors' addresses: D. Pereira (corresponding author), A. Ghose, and S. Dey, Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, West Bengal, India, 721302; emails: dannyjpereira@kgpian.iitkgp.ac.in, {anirban, soumya}@cse.iitkgp.ac.in; S. Ghosh, Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, West Bengal, India, 700108; email: sumana@isical.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

1084-4309/2023/03-ART38 \$15.00

<https://doi.org/10.1145/3576197>

platform. Each detection task deploys a state-of-the-art neural network that processes images captured through onboard camera sensors of the vehicle. This is done by applying a series of layer-wise linear algebra transformations involving a set of parameter weight tensors. The size and number of tensors vary depending on the detection task. The performance of such AI-centric edge systems is determined by the accuracy delivered for the detection task and the maximum throughput achieved while respecting individual task deadlines.

Existing Techniques: In the recent past, several works [53, 57] have proposed novel system-level real-time scheduling schemes for heterogeneous CPU/GPU platforms. These include works that

- (1) decrease the overall latency of inference tasks via software pipelining approaches to increase the detection accuracy [53],
- (2) opt for data fusion approaches followed by concurrent execution of multiple NN pipelines in the GPU [57],
- (3) use reservation-based scheduling algorithms by enabling GPU preemption [5],
- (4) use both CPU and GPU devices collaboratively with schedules decided by formulating and solving optimization problems [52] or by reinforcement learning-assisted schemes [14].

Major Limitations: The above-mentioned schemes assume that weight parameters constituting an NN pipeline remain resident on the on-chip system memory. We argue that such an assumption is not viable for real-life scenarios where there exists an inherent time-varying demand for deploying NN pipelines with different configurations of weight tensors [22]. For example, in an automotive system, an NN pipeline trained for the lane-detection task using images collected in ambient light conditions might not deliver the desired accuracy in challenging driving scenarios characterized by poor lighting, bad weather conditions, and so on. Similarly, constraints exist for other inference tasks such as pedestrian detection, traffic sign recognition, and so on. Loading and storing weights for all possible NN configurations for all inference tasks is not feasible, since the on-chip memory capacity limits embedded GPUs. Mapping schemes should be envisaged that opportunistically load/discard weight tensors as and when required while co-scheduling relevant compute operations belonging to different NN pipelines.

A Motivating Example: As a representative example, we consider the **Least Slack First (LSF)** scheduling scheme used in S^3DNN [57], a CUDA-based framework for co-scheduling multiple NN pipelines for object detection on heterogeneous CPU/GPU platforms. S^3DNN uses an orchestrating *host program* running on the CPU that employs CUDA *streams*, where each stream is an in-order command queue, to which compute/memory operations can be enqueued for actual dispatch to the GPU in the platform. An NN detection pipeline involves layer-wise transformations applied in a cascaded fashion on an input image to generate some detection result. The operations to be scheduled thus include (i) *memory operations* for copying the target image and weight tensors from the CPU host to the GPU device (H2D), (ii) *compute operations* representing a linear algebra kernel computation for each layer, and finally, (iii) *memory operations* for copying results from GPU device to CPU host (D2H).

In Reference [57], the authors assume that required weight tensors are pre-loaded in the beginning and remain resident in memory throughout. Multiple streams are instantiated to facilitate simultaneous execution of only compute operations when the target GPU has sufficient resources. We consider a memory-efficient version of Reference [57] as the baseline approach in this work, where we map both compute operations and their respective memory operations (for copying parameters). The weight tensors are fetched only when relevant compute operations require them and are discarded once their usefulness is outlived. This reduces the overall memory consumption per unit of time on the target platform. The dispatching order for both compute and memory operations are prioritized based on their slack values, i.e., the allowable time left to finish the operation

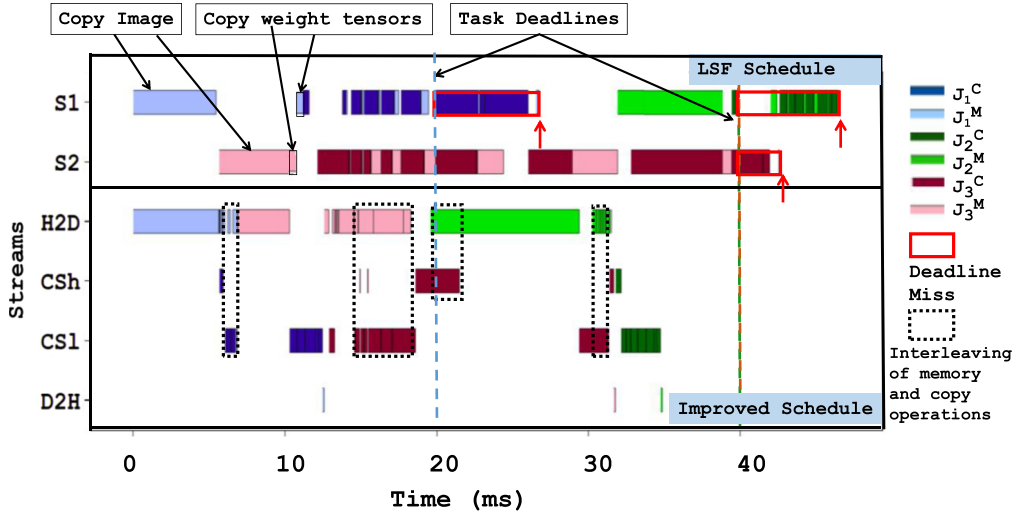


Fig. 1. Gantt charts for different scheduling schemes.

in question and other pending operations in the same pipeline within its deadline. Considering the NVIDIA Jetson NX as our target platform, we use the LSF scheme to co-schedule two periodic NN pipelines (i) T_1 : YoloLite [18] and (ii) T_2 : tinyYolov2-VOC [42] with T_1 arriving at an interval of 20 ms and T_2 arriving at an interval of 40 ms. The upper Gantt chart in Figure 1 depicts the corresponding schedule produced for one hyper-period of 40 ms, which comprises two jobs J_1 and J_2 pertaining to T_1 and one job J_3 pertaining to T_2 . We employ two CUDA streams S_1 and S_2 (depicted in the y-axis of the top half of Figure 1) for processing the compute and memory operations constituting the jobs of T_1 (J_1 and J_2) and T_2 (J_3), respectively. For each job J_i , we fix a particular color and depict deadlines by vertical lines of the same color. The rectangles representing memory and compute operations of J_i are colored with lighter and darker shades of the designated color, respectively. We label compute and memory operations in each job J_i as J_i^M and J_i^C , respectively, and provide the color encoding used in the legend of Figure 1.

Although the scheme allows for loading different NN weights as and when required, this is achieved at the cost of all jobs missing their deadlines (operations missing their deadlines are highlighted by a red border in Figure 1). One can observe that even though the memory operation for copying the image for job J_1 is enqueued on S_1 first, its subsequent operations are delayed by the memory operations for copying the image and weight tensors of job J_3 enqueued on S_2 . This is because memory operations can execute one at a time due to a single H2D copy engine in the embedded platform. The scheme cannot enforce the correct priority to the respective operations, causing J_1 to miss its deadline.

We believe an *improved operation-to-stream schedule* can be designed by instantiating dedicated streams for memory operations and compute operations rather than individual streams for operations about a given pipeline. Furthermore, CUDA facilitates enforcing two priority levels (high and low) in streams servicing compute operations. If multiple compute operations cannot be simultaneously executed, then we can ensure that compute operations dispatched from a high-priority stream would complete before operations dispatched from a low-priority stream. Keeping these observations in mind, we represent an improved schedule in the Gantt chart in the lower half of Figure 1. One can observe from the y-axis in this Gantt chart that we use streams $H2D$ and $D2H$ for servicing only H2D and D2H operations, respectively. The streams CS_h and CS_l represent high-priority and low-priority streams servicing only compute operations. Using such a combination

of streams, one can observe from the dotted rectangles in Figure 1 that compute operations can execute concurrently with memory operations. For example, in the first dotted rectangle, H2D transfers for later layers in job J_1 can execute simultaneously with the compute operations in its earlier layers, allowing J_1 to finish considerably before the deadline. In contrast to the LSF schedule, the improved schedule reports no deadline misses. Although the *improved operation-to-stream schedule* ensures that timing requirements of jobs are met without having to store weight tensors throughout the duration of the schedule, one can observe from the Gantt chart at the bottom of Figure 1 that: (i) weight tensors associated with compute operations are prefetched and stored earlier than required; (ii) jobs in general finish considerably before their deadlines. For example, in the case of job J_2 , which has a deadline of 40 ms, all H2D operations (prefetching weight tensors) are completed before 20 ms while its last compute operation finishes at 32 ms. The aggressive prefetching of weight tensors increases the overall memory consumption per unit of time, since those tensors cannot be relinquished from memory until and unless their associated compute operations have finished execution. Given the available idle slots in each of the respective streams and the fact that tasks finish considerably before their deadlines, one can sufficiently adjust the start and finishing time of operations so the overall memory consumption per unit time is also decreased. Our proposed work investigates scheduling mechanisms that not only optimize with respect to deadline requirements of inference tasks but also with respect to memory utilization in resource-constrained embedded platforms.

Our Proposal: While mechanisms for enqueueing operations to dedicated streams look promising, deriving a real-time, memory-efficient scheduling solution entails examining every possible mapping decision that considers the tasks' computation overheads, memory footprints, and deadline requirements. This requires searching through a considerably large solution space where the underlying combinatorial structures for finding an optimal assignment of operations to dedicated streams need modeling both linear and first-order logic constraints (e.g., use of existential quantifier). In this regard, we employ a **Satisfiability Modulo Theory (SMT)-based optimizer** [33] to synthesize a scheduling solution that minimizes the response times of each NN-based detection pipeline running on heterogeneous embedded GPGPU platforms. The optimizer essentially solves a constraint satisfaction problem in an offline manner, producing a set of scheduling decisions for mapping compute and memory operations to different compute and memory streams, respectively. This is followed by a multi-pass memory optimization algorithm that adjusts the operation start and finish times in the schedule produced by the SMT optimizer to improve memory utilization.

We note that our proposed method is useful for any real-time task set with fixed periodic behavior, i.e., for each task, the data to be processed arrives once inside a period and accordingly one task instance of each task type must execute once inside the specified period of the task. A popular use case in this regard is **Advanced Driver Assistance Systems (ADAS)**. In such systems, multiple object detection tasks (pedestrian detection, traffic light detection, traffic detection, etc.) need to run and process image frames captured by one or multiple cameras. Depending on the criticality, object of interest, and image source, the period of such tasks may be set accordingly. Surely, the lowest period is determined by the max frame rate supported by the camera system. Given a resource-constrained platform to process such tasks, our proposed edge device scheduling schemes ascertain suitable operation-to-stream mapping decisions that can sustain inference task execution at higher rates, decreasing the response time and subsequently increasing the detection accuracy.

Proposed System Overview: The overall system architecture for our problem is depicted in Figure 2. We consider as input a set of NN pipelines represented in the form of a **directed acyclic graph (DAG)** capturing the dependency information between H2D, kernel, and D2H operations in the pipeline. For each DAG, the operation-level profile times as well as the period specification, i.e., the deadline requirements, are provided. Given this and the type/number of streams used in the

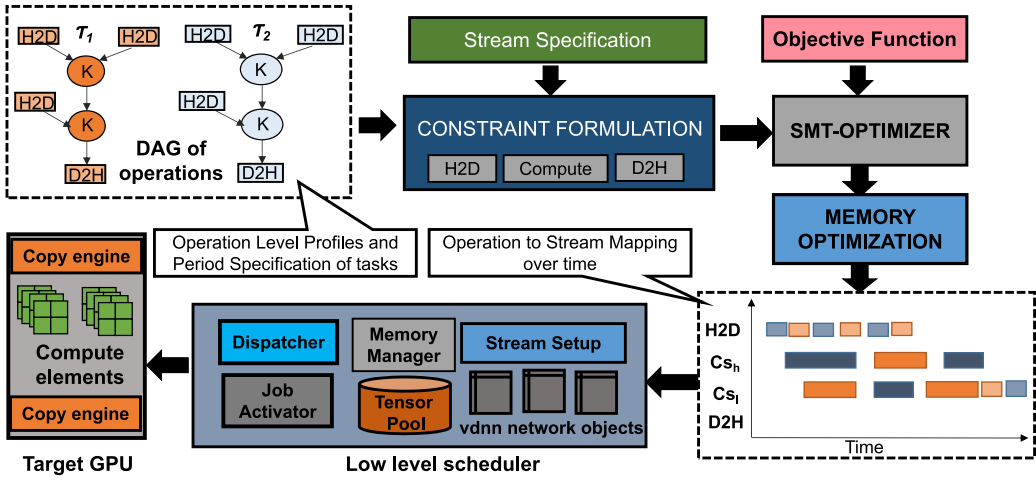


Fig. 2. Proposed system overview.

target platform, the set of constraints involving all types of operations belonging to the DAGs are next formed. For our proposed work, we consider the four streams ($H2D$, $D2H$, CS_h , CS_l) depicted in the lower Gantt chart of Figure 2. The formulated constraints are passed as input to the SMT optimizer along with a relevant objective function targeting some performance metric. The output of the SMT-optimizer is an operation-to-stream mapping for the periodic DAGs in one hyper-period. Our proposed work is the first to consider asynchronous overlapping of memory and compute operations belonging to multiple inference applications on resource-constrained embedded GPGPU platforms while accounting for different priority options supported for dispatching commands.

The operation-to-stream mapping produced by the SMT optimizer is passed through a memory optimization pass that changes the start and finish times of operations to improve memory utilization. We validate the final schedule obtained on a low-level scheduler that is implemented on top of vDNN++ [47], a popular runtime framework that facilitates automatic memory management while training NN pipelines. The scheduler sets up the required memory and compute streams with priority levels and NN objects pertaining to the inference tasks. Following the period specification of the DAGs, a *job activator* routine synthesizes a steady stream of jobs arriving for execution. The *dispatcher* processes the operation-to-stream mapping reported by the SMT-optimizer for a job that has arrived, enqueues relevant operations to streams, and initiates execution on the target GPGPU edge platform. We enumerate the salient features of our proposed work as follows:

- (1) **Formulation:** We formalize an SMT-based formulation by designing relevant constraints that capture how memory operations and compute operations should be interleaved while scheduling NN pipelines with deadline constraints on embedded GPGPU platforms.
- (2) **Implementation:** We create a memory-efficient low-level scheduler capable of setting up required streams and enqueueing the operations as dictated by the schedules reported by the SMT optimizer.
- (3) **Memory optimization:** We implement a multi-pass memory optimization algorithm, which takes the SMT schedule as input and modifies the ordering of operations to reduce the overall memory footprint.
- (4) **Validation:** We validate the usefulness of our SMT-assisted scheduling approach and memory optimization algorithms through extensive experimentation by using real-life object detection pipelines on the Jetson Xavier NX board, a popular AI-centric edge platform.

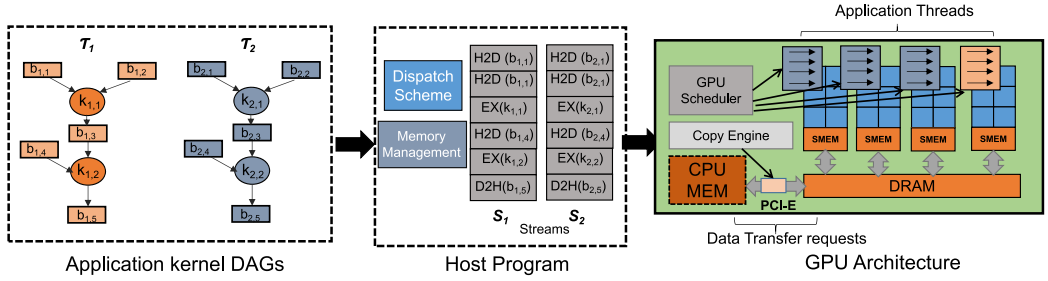


Fig. 3. GPGPU scheduling.

2 BACKGROUND AND PRIOR WORK

In this section, we provide a brief overview of the GPGPU programming model, followed by relevant literature on real-time scheduling techniques and memory optimization techniques proposed in the context of deep learning workloads over recent years.

2.1 Heterogeneous GPGPU Programming Model

The GPGPU programming model [12] allows for General Purpose Computation on GPUs, i.e., apart from graphics workloads, data-parallel style compute workloads can also be executed on GPUs. We restrict our discussion to NVIDIA GPU architectures and, subsequently, the execution model offered by CUDA [34], the programming framework for designing data parallel applications on NVIDIA GPUs. Applications implemented in CUDA comprise two distinct programming entities: (i) a host-side program executing sequentially on a CPU core and (ii) device-side computational kernel(s) executing on the target GPU. The overall workflow in mapping CUDA applications to NVIDIA GPU architectures in the GPGPU programming model is depicted in Figure 3.

In the left-hand side of Figure 3, two **directed acyclic graphs (DAGs)** τ_1 and τ_2 are depicted where the vertices of the DAG include **kernels** (represented by circular nodes and labeled with k -prefix) and their associated **buffers** (represented by rectangular nodes and labeled with b -prefix). Each kernel consumes a set of input buffers, applies a specific data parallel operation on the data resident in the buffers, and produces an output buffer to be processed by another kernel. The dependency information between kernels and associated buffers in each DAG dictates the precedence constraints between the constituent kernels. As depicted in Figure 3, the host program implementation must provide (i) **memory management of buffers** and (ii) a **dispatching scheme for copy and compute operations**. In Figure 3, the host program maintains two streams, S_1 and S_2 , for processing the copy and compute operations of DAGs τ_1 and τ_2 , respectively. The enqueueing order in each stream S_i ensures that *copy operations* (H2D and D2H) and *execute operations* (EX) are enqueued following some topological ordering of the vertices in each DAG τ_i . As depicted in the right-hand side of Figure 3, the GPU device consists of an array of **streaming multiprocessors (SMs)**, each SM having a dedicated shared memory. The constituent SMs share a **global on-chip DRAM memory**. The copy engine copies the required data between the CPU memory space and the GPU side global memory via the PCI-Express bus. Depending on the availability of SMs and memory resources, the GPU hardware scheduler can overlap the execution of multiple kernels enqueued in different streams. We further note that the scheduler is capable of kernel pre-emption subject to the associated priorities of the kernels being dispatched. Currently, the CUDA programming standard provides two levels of priorities (high and low) for streams, and kernels enqueued in streams with a **high (low) priority attribute** are associated with a **high (low) priority** by the underlying scheduler. High-priority kernels can preempt low-priority kernels by the GPU

scheduler subject to the execution status of the constituent threads of the kernels executing on the SMs [1, 5, 36]. In our proposed work, we exploit this feature and design a scheduling framework that can efficiently overlap copy and compute operations while conforming to individual deadline requirements of multiple kernel DAGs.

2.2 SMT Optimizer

Satisfiability Modulo Theories (SMT) is an extension of the **Boolean Satisfiability Problem (SAT)**. An SMT problem instance is a first-order logic formula with predicates and function symbols to be interpreted for a given *theory*, such as the theory of reals, integers, bit-vectors, and so on. The decision problem is to check whether the formula is satisfiable or not. The input to an SMT-solver is the set of SMT variables, constants, and constraints written in first-order logic. The output is the solution in terms of value assignments to the SMT variables. Modern solvers like Z3 also provide an optimization mode using SMT techniques. Given a set of SMT variables and constraints along with an objective function, *SMT-optimizers* provide a solution in terms of value assignment to all SMT variables such that the objective function is maximized/minimized. Thus, while an SMT-solver gives a single satisfiable solution in a run for a decision problem, an SMT-optimizer provides an optimized solution. For a comprehensive background on SMT, please refer to Reference [2].

2.3 Prior Work

As discussed earlier, object detection pipelines employed in ADAS leverage **Convolutional Neural Network (CNN)** architectures, which is the de facto standard for image recognition tasks [13, 26, 51]. In general, any CNN architecture is represented by a set of parameter tensors learned in the training phase and a sequence of linear algebra operations such as convolution, pooling, matrix multiplication, and so on, which is applied in a cascading fashion to some input image to obtain a classification result. The most notable set of CNN architectures used in the context of object detection in ADAS includes the **You-Only-Look-Once (YOLO)** family (YOLOv1 [41], YOLOv2 [42], and YOLOv3 [43]). While such CNN architectures can predict objects in the vicinity with a high level of accuracy, delivering real-time detection results is difficult owing to the high resource demand imposed by them for execution on embedded devices. Keeping this in mind, several lightweight CNN architectures such as MobileNet [17], SqueezeNet [20], tinyYOLO [42, 43], YOLO-LITE [18], PVANET [25] have been proposed in the recent past, which can be readily deployed on resource-constrained embedded GPGPU boards.

AI frameworks for Inference: Frameworks such as Intel OpenVino [9], NVIDIA Drive Perception [35], Tensorflow-lite [48], and so on, are typically used for designing and rapid deployment of CNN-based object detection pipelines. In the recent past, several dedicated object recognition systems enhanced with optimizations for efficient deployment of CNNs on mobile devices have also been proposed. Glimpse [7] is a real-time object recognition system that intelligently offloads specific video frames to a server to obtain object recognition hints. DeepMon [19] is an edge-centric recognition system for resource-constrained mobile devices that employs an intelligent caching mechanism of video frames for reducing the memory footprint of intermediate outputs generated while executing CNNs. DeepSense [54] also presents an edge-centric recognition system with a specific focus on algorithmic optimization for object detection tasks using a combination of CNNs and **Recurrent Neural Networks (RNNs)**. In Reference [23], the authors proposed graph optimization techniques to optimize the execution of inference pipeline on edge devices efficiently. The common objective for all frameworks mentioned above lies in either algorithmic optimizations or input frame-level caching optimizations for accelerating the execution of individual pipelines. Although these frameworks offer a scheduling component, they are not equipped to handle the

execution of multiple detection pipelines while maintaining their individual real-time guarantees. This requires synthesizing intelligent operation-level dispatching decisions for a target platform. We will discuss the most relevant works in this regard next.

Real-time GPGPU Scheduling: The work reported in Reference [53] proposed a pipeline-based mechanism for increasing the throughput of CNN execution on embedded GPUs. The proposed system argues that traditional CNN-based detection systems must be redesigned to handle multiple image streams obtained through multiple onboard camera sensors to understand the vehicular environment better. This is done either by (i) forming a single composite image of the frames captured through different cameras and using a global classifier that would predict the required objects of interest in all the input frames simultaneously or (ii) using pipeline-based parallelism and concurrent kernel execution via streams to optimize the processing of a CNN over multiple frames.

The work reported in Reference [57] proposed **S³DNN (Supervised Streaming and Scheduling algorithm for DNNs)**, which adapts classical real-time least-slack-first scheduling for concurrent kernel execution using streams on GPU devices. The proposed approach in this work takes multiple video streams as input, applies data fusion techniques to the frames obtained, and schedules multiple **Deep Neural Network (DNN)** instances to process the fused frames. The primary assumption in both the works mentioned above is that the parameter tensors of the CNNs are preloaded into the system-on-chip memory beforehand. The work reported in Reference [5] presents a prototype **Earliest Deadline First (EDF)**-based scheduler implementation with support for resource reservation. The prototype is essentially an event-driven scheduler running as a module in the NVIDIA software stack, providing finer preemption granularity for kernel compute operations. The scheduling decisions do not consider potential overlapping opportunities that may exist while dispatching copy and compute operations.

Collaborative CPU-GPU Execution: The work reported in Reference [52] proposed DART, a real-time scheduling framework of DNN tasks for heterogeneous embedded platforms. In contrast to works discussed earlier, DART views target resources as a collection of CPU and GPU devices. Given a DNN instance, a dynamic programming-based optimization approach is applied to determine a stage-level decomposition of the DNN layers where each stage comprises a consecutive set of layers. After this, the framework determines an optimal allocation of stages to underlying CPU and GPU devices while ensuring that the worst-case response time of each detection task respects its deadline constraints. The work reported in Reference [14] developed a combined AI and control theory-based approach for the real-time scheduling of CNN pipelines on heterogeneous CPU/GPU platforms. Like DART, the proposed method determines the stage-level decomposition of object detection tasks and obtains the optimal device allocations. However, this approach also accounts for dynamic resource requirements subject to the driving context and subsequently uses Reinforcement Learning to determine appropriate mapping decisions. Once deployed on the edge device, the mapping decisions are continuously monitored by a controller that senses potential deadline violations and applies core-level DVFS on the fly as and when required. We note that all reported works in this area primarily consider scheduling compute-level operations, i.e., they assume that weight tensors needed for each layer are already resident in the on-chip memory at the time of scheduling. In contrast, our proposed work investigates how memory-bound operations can be scheduled asynchronously alongside compute operations, thereby decreasing the overall memory footprint per unit of time.

Optimizing Memory Consumption in GPU: In the recent past, an array of works has been proposed, targeting the reduction of memory consumption while training neural networks on high-performance GPUs. The work reported in Reference [44] proposed a framework called **virtualized DNN (vDNN)** that allows training of large DNNs, where all tensors of the underlying neural

network architecture cannot be resident together inside the on-chip GPU memory. In Reference [47], the authors improvised and proposed *vDNN++* to eliminate the shortcomings of memory fragmentation and delayed computation startup times in *vDNN*. In a similar vein, the work reported in Reference [8], proposed **memory optimal Deep Neural Network Training (*moDNN*)**, which further extends upon the optimizations proposed in *vDNN* and *vDNN++* by exploring methods for asynchronously interleaving copy and compute operations during the forward and backward stages of the training workflow. There also exists a number of works [39, 49, 50] that explores memory optimizations in the context of distributed learning of DNNs.

While considerable works exist for optimizing memory consumption in the context of DNN training, some works also address the issue of memory bottleneck during inference. The authors in Reference [27] proposed methods to optimize memory consumption in large CNNs by investigating the impact of data layout and memory access pattern on memory consumption. In Reference [38], authors provided strategies to reduce memory consumption by sharing memory buffers among intermediate tensors. The work reported in Reference [21] proposed incremental weight tensor loading and data layout reorganization for memory optimization in deep inference pipelines. The framework *TrIMS* [10] is designed for cloud servers where Deep Learning models are shared across multiple isolated functions, minimizing model loading and data movement overhead. We note that all approaches discussed so far consider optimizing on-chip memory consumption for the training/inference stages of a single DNN workflow. In Reference [15], the authors investigate architectural designs (**Shared Virtual Memory (SVM)**) and collaborative execution patterns (such as data partitioning and task partitioning) to design benchmark suites for evaluating heterogeneous platforms. In Reference [4], authors proposed a task-level runtime scheduler that orders the incoming tasks as per **Earliest Deadline First (EDF)** policy and decides the GPU memory management methods (Device Memory, Managed Memory, Host-Pinned Memory) for each task to reduce the overall memory footprint and latency. Here, scheduling decisions are taken at the task level rather than the operation level, as in our case. The experiments were conducted on single-kernel tasks and not on multiple inference pipelines. We further note that in References [4, 10, 15], **Unified Virtual Memory (UVM)** has been used for managing memory. While UVM reduces code complexity by supporting implicit memory management schemes, this reduction comes at the cost of performance. Such approaches are suitable for applications where memory access patterns are predictable. For our use case, where multiple periodic applications are scheduled and the memory consumption at runtime is subject to the arrival pattern of jobs, explicit API calls for memory management make more sense. In this context, we propose a low-level scheduler that automatically allocates/deallocates memory by invoking such API calls without requiring intervention from the user during the scheduling process. Our schemes judiciously shift memory and compute operations while co-scheduling multiple inference pipelines so the overall memory footprint imposed by the execution of each pipeline remains within a user-supplied memory budget.

Profile-guided scheduling: In general, real-time scheduling of tasks on multi-core machines is NP-complete [32, 40]. Profile-guided scheduling methods entail profiling resource usage (CPU, GPU, memory, etc.), measuring the total execution time of compute tasks, memory tasks, and so on, during the execution of the applications and using this information while scheduling. In Reference [28], authors proposed a latency prediction model for predicting execution times of layers in a convolutional neural network. In Reference [52], layer-wise execution time profiles involving operation execution times and data transfer times are used to devise the real-time scheduling framework *DART*. In Reference [24], the authors proposed a scheduling mechanism that uses profiling data collected offline for DNNs, including execution times on CPU/GPU, memory transfer times between CPU and GPU, and the time taken for quantization/de-quantization operations. The work reported in Reference [31] uses execution time history as profiling information to efficiently

manage power consumption in GPUs based on **Model Predictive Control (MPC)** techniques. In References [29, 30], the authors propose thermal-aware real-time scheduling mechanisms that rely on adaptive scheduling decisions based on latency models built offline by profiling extensively on the target platforms. In our work, we have profiled execution times at the operation level for all the inference pipelines. We also conducted profiling passes that characterize the co-schedulability of multiple operations across streams of varying priority.

3 PROPOSED SOLUTION FRAMEWORK

This section presents a brief overview of the system model followed by a detailed description of the SMT-assisted offline solution approach.

3.1 System Model

Target Platform: We consider a target heterogeneous CPU/GPU platform, with the set of streams $\{M_{h2d}, M_{d2h}, CS_h, CS_l\}$. Here, M_{h2d} and M_{d2h} are dedicated streams for servicing H2D and D2H copy operations, while CS_h and CS_l represent high-priority and low-priority streams for compute operations in each NN pipeline. The limited number of compute units on edge devices limit the available concurrency. Hence, we have considered only two compute streams with high and low priority.

Task Model: Every NN-based inference task contains a sequence of layers where (1) the first layer always involves two H2D *memory* operations (one for copying input image, one for copying weights), followed by one *compute* operation; (2) the intermediate layers involve one H2D *memory* operation for weights followed by one compute operation; and (3) the last layer involves H2D and compute operations similar to an intermediate layer with an additional D2H operation to copy back the detection result to the host CPU. We consider the popularly used linear feedforward style **Convolutional Neural Networks (CNNs)** for our work. The approach can easily be generalized for other graph structures as well.

For an l -layer CNN detection pipeline, we have in total $p = 2l + 2$ operations where 0th and every odd indexed operation is an H2D memory operation, while every even indexed operation is a compute operation. The last, i.e., $(p-1)$ -th operation, is a D2H memory operation. We model a CNN inference task τ_i as a DAG $G_i = \langle V_i, E_i \rangle$, where V_i represents the set of operations (memory and compute) to be run on the target GPU device, and E_i contains the collection of edge dependencies between the operations that encode the precedence constraints of the DAG. We consider a system-setting, $\langle T, H, \mathcal{D} \rangle$, where $T = \langle \tau_1, \tau_2, \dots, \tau_n \rangle$ is the set of n tasks, $H = \langle h_1, h_2, \dots, h_n \rangle$, and $\mathcal{D} = \langle D_1, D_2, \dots, D_n \rangle$ are the set of *periods* (i.e., set of fixed time intervals at which tasks arrive) and the relative deadlines of those tasks, respectively. For a fixed system setting, the *hyper-period* $\mathcal{H}$ is computed as the **least common multiple (lcm)** of periods, i.e., $\text{lcm}(h_1, h_2, \dots, h_n)$. A task τ_i has a total $\mathcal{J}_i = \mathcal{H}/h_i$ job instances inside the hyper-period $\mathcal{H}$. We denote the k th job of τ_i as $J_{i,k}$. The activation time of a job $J_{i,k}$ is represented as $a_{i,k}$, its absolute deadline as $d_{i,k} = a_{i,k} + D_i$, and its finish time as $f_{i,k}$. The response time of $J_{i,k}$ is the time elapsed between its arrival and finish time, i.e., $f_{i,k} - a_{i,k}$. Hence, for a task τ_i , its **Worst-Case Response Time (WCRT)** is $R_i = \max_k \{f_{i,k} - a_{i,k}\}$ (i.e., maximum among all its jobs), and τ_i is said to be *schedulable* if $R_i \leq D_i$.

A task τ_i has p_i operations in total. Since every odd indexed operation along with the 0th (i.e., first) and $(p_i - 1)$ -th (i.e., last) operations are memory operations, we define the set $\mathcal{M}_{op}^i = \{0\} \cup \{2q - 1 \mid 1 \leq q \leq \frac{p_i}{2}\}$ containing all these indices. Similarly, since every even indexed operation is a compute operation, we define the set $\mathcal{C}_{op}^i = \{2q \mid 1 \leq q \leq \frac{p_i-2}{2}\}$. We use the triplet (i, k, x) to indicate the x th operation within the k th job of the i th task. For the sake of simplicity, we refer (i, k, x) as an *operation* respecting its meaning as specified above. For example, we consider two

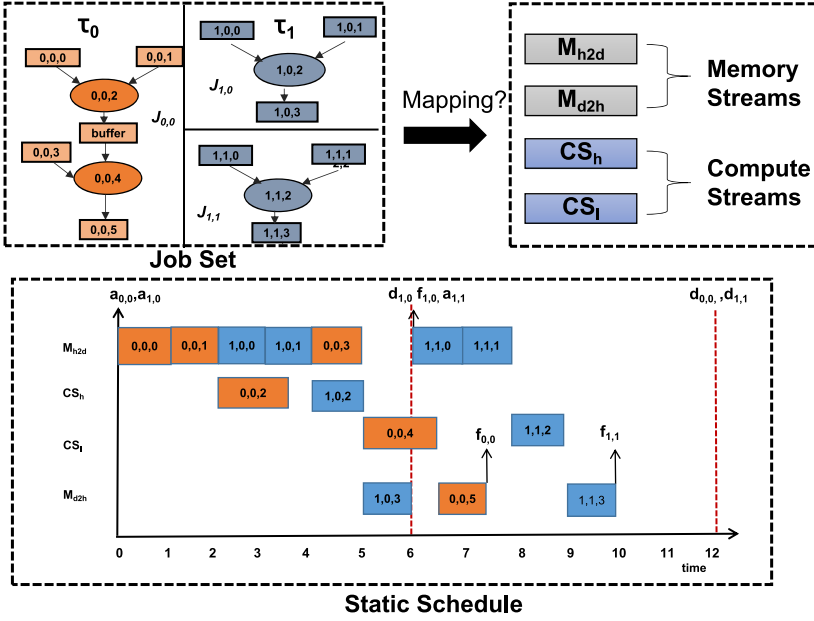


Fig. 4. Operation to resource mapping.

CNN tasks τ_0 and τ_1 where τ_0 comprises two layers, and τ_1 comprises one layer. Figure 4 shows the pictorial view of τ_0 and τ_1 with one job instance, $J_{0,0}$, of τ_0 and two job instances, $J_{1,0}$ and $J_{1,1}$, of τ_1 . Let the periods for tasks τ_0 and τ_1 be $h_1 = 12$ and $h_2 = 6$, respectively. Then their hyper-period $\mathcal{H}$ is $lcm(12, 6) = 12$. Each constituent operation of the jobs is annotated with (i, k, x) labels, as discussed above.

Underlying Optimization Problem to Solve: Given a task set $\mathcal{T}$ with the hyper-period $\mathcal{H}$, our objective is to ascertain:

- (1) the start and finish times of every operation of all jobs of all tasks within $\mathcal{H}$, and
- (2) mapping of memory operations to the memory stream and compute operations to the high/low-priority streams such that weighted sum of WCRTs, i.e., $\sum_{\tau_i \in \mathcal{T}} w_i \cdot R_i$, is *minimized* while respecting deadlines of all the tasks.

Here, R_i and w_i represent the response time and weight of τ_i , respectively, and we set $w_i = 1/h_i$, i.e., the lesser the period, the higher the weight. In other words, we aim to obtain a *static schedule* containing *feasible* assignments to start time and finish times as well as target streams for every operation while *minimizing* the weighted WCRTs of all the tasks. One example of a static schedule is depicted in the lower part of Figure 4. Note that a static schedule is said to be *feasible* if all the tasks finish execution before their corresponding deadlines, i.e., $\forall \tau_i \in \mathcal{T}, R_i \leq D_i$.

Towards the goal of minimizing WCRTs of all the tasks while respecting deadline constraints, one should allow *maximum overlap* among the operations, which can be achieved in two ways:

- (1) maximum overlap between memory operations and compute operations,
- (2) maximum overlap (i.e., co-scheduling) of compute operations on high- and low-priority compute streams.

Now, the solution to such an optimization problem requires examining every possible mapping decision that considers the tasks' resource utilization and deadline requirements. Again, this

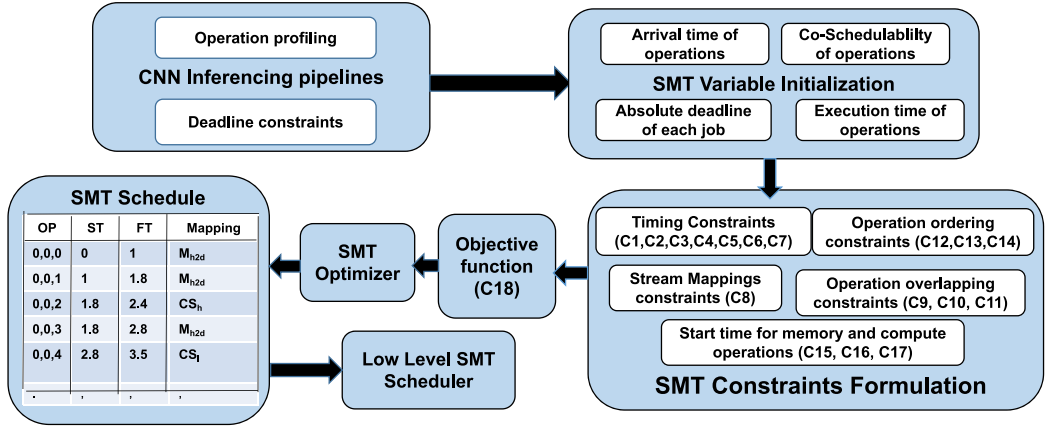


Fig. 5. The workflow.

requires searching through a considerably large solution space and capturing the complex behavior of various overlapping scenarios through modeling both the linear and first-order logic constraints (e.g., use of existential quantifier). For this reason, we employ an SMT-based solution framework for our purpose.

3.2 Work Flow Overview

The overall workflow of the proposed solution framework is illustrated in Figure 5. We consider as input a collection of n CNN tasks, $T = \langle \tau_1, \tau_2, \dots, \tau_n \rangle$, as described in Section 3.1. A task τ_i is given with a set of information, $\langle D_i, (a_{i,1}, \dots, a_{i,J_i}), (e_{i,1}, \dots, e_{i,J_i}) \rangle$, where D_i is the deadline of τ_i , $a_{i,k}$ and $e_{i,k}$ are the arrival time and the respective execution time estimate of the k th job of τ_i . Note, τ_i has total J_i jobs, as discussed in Section 3.1. All this timing information is used in subsequent steps to develop the state space of the SMT-based solution framework. The next step is the *SMT variable assignment and initialization* phase. We define SMT variables for capturing (1) timing information such as arrival, start, and finish times of operations, absolute deadlines of jobs, and so on; (2) stream mapping information; (3) ordering sequence of multiple operations in the same stream; (4) overlapping and co-schedulability information of multiple compute operations. A detailed description of these SMT variables is presented in Section 3.3. The next step is developing the *SMT-constraint formulation* that sets up desired constraints capturing stream mapping decisions, timing conditions, operation orders, and different overlapping scenarios. The constraints considered in our formulation can be broadly categorized as follows:

- (1) *Timing Constraint*: Specifies upper bounds for arrival, start, and finish times of all operations.
- (2) *Mapping Constraint*: Decides the mapping of compute operations to either low- or high-priority streams.
- (3) *Operation Overlapping Constraint*: Identifies whether the time intervals of two operations overlap or not.
- (4) *Operation Ordering Constraints*: Specifies ordering of operations in or across streams to identify the immediately preceding operation of an operation under consideration. Note that this would be subsequently used to determine the start time of any operation in a stream.

We define 17 such constraints, $C1, C2, \dots, C17$, under these four categories. A detailed description of each of these constraints is presented in Section 3.4. For clarity, respective constraint IDs are added in Figure 5 for each category. Next, we define the constraint for the objective function

(C18) and feed this to the SMT-optimizer along with the other 17 constraints for the solution. The SMT optimizer produces an optimal static schedule comprising all operation start and finish times together with their target stream allocation. Finally, this static schedule is directed to the low-level scheduler for processing and implementation over the target platform.

3.3 Defining SMT variables

We first introduce the Boolean and real variables used in our SMT formulation.

V1. Stream Mapping Variables: We define two Boolean variables $b1_{i,k,x}$ and $b2_{i,k,x}$ for each compute operation to indicate if the operation (i, k, x) is assigned to either high- or low-priority CUDA stream, respectively. Note that a compute operation can only be mapped to one stream at any time point; e.g., if operation (i, k, x) is assigned to the high-priority stream, then we set $b1_{i,k,x}$ to *true* and $b2_{i,k,x}$ to *false*. In this article, we use the notation b and $\neg b$ to indicate the *true* and *false* values of a Boolean variable b , respectively.

V2. Timing Variables: Next, we define two real variables $ST_{i,k,x}$ and $FT_{i,k,x}$ for each operation (i, k, x) to indicate its start time and finish time, respectively. Since these variables reflect a notion of time, we bound their domain as: $ST_{i,k,x} \geq 0 \wedge FT_{i,k,x} \geq 0$.

V3. Variables for Operation Ordering: The following variables are defined to encode the order of memory and compute operations in respective streams. For all possible pairs of memory operations $((i_1, k_1, x_1), (i_2, k_2, x_2))$ where $1 \leq i_j \leq n$, $1 \leq k_j \leq \mathcal{J}_{i_j}$, and $x_j \in \mathcal{M}_{op}^{i_j}$, $j = 1, 2$, the Boolean variable $IP_{M(i_1, k_1, x_1)(i_2, k_2, x_2)}$ when set to *true* denotes that (i_1, k_1, x_1) immediately precedes (i_2, k_2, x_2) in the memory stream. Similarly, to identify the order of compute operations (i_1, k_1, x_1) and (i_2, k_2, x_2) where $1 \leq i_j \leq n$, $1 \leq k_j \leq \mathcal{J}_{i_j}$, and $x_j \in \mathcal{C}_{op}^{i_j}$, $j = 1, 2$, we define the following Boolean variables. The variable $IP_{L(i_1, k_1, x_1)(i_2, k_2, x_2)}$ is set to *true* if (i_1, k_1, x_1) immediately precedes (i_2, k_2, x_2) in the low-priority compute stream. The variable $IP_{H(i_1, k_1, x_1)(i_2, k_2, x_2)}$ is set to *true* if (i_1, k_1, x_1) immediately precedes (i_2, k_2, x_2) in the high-priority compute stream.

V4. Variables for Overlapping Operations: Operations from different compute streams can execute concurrently. For each possible pair of *different priority bound compute operations* (i_1, k_1, x_1) and (i_2, k_2, x_2) , if their execution intervals overlap, then the variable $O_{(i_1, k_1, x_1)(i_2, k_2, x_2)}$ is set to *true*. Otherwise, it is set to *false*. We also define another variable $LO_{H(i_1, k_1, x_1)(i_2, k_2, x_2)}$, which determines whether the high-priority compute operation (i_1, k_1, x_1) is the last operation to overlap with the low-priority compute operation (i_2, k_2, x_2) . This would be subsequently used to determine the start times of low-priority operations (discussed later), since we know that when operations are overlapped, high-priority operations would be given resources earlier and may finish earlier than low-priority operations.

Input Variables: We define the following input variables whose values are input to our SMT-based framework:

- (1) Two real variables $a_{i,k,x}$ and $C_{i,k,x}$ for each operation (i, k, x) to indicate its arrival time and execution time, respectively.
- (2) Two real variables $a_{i,k}$ and $D_{i,k}$ to indicate the arrival time and absolute deadline of the k th job of i th task, respectively.
- (3) To model the *co-schedulability condition* of two *different priority bound compute operations* (i_1, k_1, x_1) and (i_2, k_2, x_2) , we use a set of Boolean variables. The variable $S_{(i_1, k_1, x_1)(i_2, k_2, x_2)}$ is set to *true* if (i_1, k_1, x_1) and (i_2, k_2, x_2) can be co-scheduled together in different streams. Otherwise, it is set to *false*. Simultaneously executing kernels using multiple streams may not always lead to better performance due to memory interference, lack of special functional units, and so on. To characterize this, each possible pair of kernels belonging to the pipelines to be scheduled are launched in different compute streams (in both possible mapping options

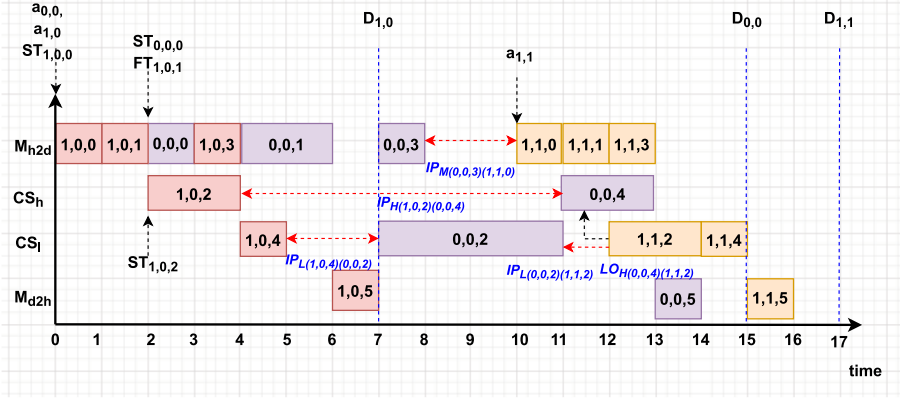


Fig. 6. A sample SMT-schedule.

w.r.t. priority), and the time to execute both kernels concurrently is measured. For each kernel pair, we calculate the ratio of concurrent execution time over total sequential execution time (i.e., time taken to run both kernels sequentially in isolation). If we observe that the resultant speedup is greater than 80%, i.e., there exists sufficient speedup, then we set the Boolean variable for the given kernel pair to *true*, else *false*.

3.4 Developing SMT Constraints

We formulate the set of constraints required for modeling the optimization problem as follows: As mentioned in Section 3.1, we have n tasks in total where the i th task has $\mathcal{J}_i$ jobs and p_i operations. For illustration purposes, we consider a running example of an SMT schedule depicted in Figure 6. The task configuration of this example is as follows: The first task has one job, and the second has two jobs. The start time, along with the mapping of compute and memory operations to four streams, is depicted in the figure. The arrival time of the single job of the first task is $a_{0,0} = 0$, whereas the arrival time of the first job and second job of the second task are $a_{1,0} = 0$ and $a_{1,1} = 10$, respectively. The absolute deadlines for these three jobs are $D_{0,0} = 15$, $D_{1,0} = 7$, and $D_{1,1} = 17$.

C1. Arrival Time of Memory Operations: A memory operation within a job of a task arrives only when the memory operation of the previous job of that task gets over. Formally,

$$\forall i, 1 \leq i \leq n, \forall k, 1 \leq k \leq \mathcal{J}_i, \forall x \in \mathcal{M}_{op}^i, a_{i,k,x} = \max(FT_{i,k-1,p_i-1}, a_{i,k}).$$

Here, $FT_{i,k-1,p_i-1}$ is the finish time of the $(k-1)$ -th job. The function $\max(a, b)$ is implemented using the *if-then-else* construct of the SMT-solver, i.e., *If* ($a \geq b$) *assign* a ; *else assign* b . Recall that $\mathcal{M}_{op}^i$ contains indices of all memory operations of τ_i .

C2. Arrival Time of Compute Operations: A compute operation (i, k, x) can arrive only when its immediate previous memory operation $(i, k, x-1)$ and compute operation $(i, k, x-2)$ finish their executions. Hence,

$$\forall i, k, 1 \leq i \leq n, 1 \leq k \leq \mathcal{J}_i, \forall x \in \mathcal{C}_{op}^i, a_{i,k,x} = \max(FT_{i,k,x-1}, FT_{i,k,x-2}).$$

Note that $\mathcal{C}_{op}^i$ contains indices of all compute operations of τ_i .

C3. Lower Bound of Start Time of Memory Operations: Recall that 0th and all odd-indexed operations are H2D transfers. The first memory operation (i.e., 0th) of weight parameters can start immediately after task arrival time, i.e., $(ST_{i,k,0} \geq a_{i,k,0})$. The first memory operation happens with the 0th finishing, i.e., $(ST_{i,k,1} \geq ST_{i,k,0} + C_{i,k,0})$. All other memory operations can start only

when its preceding memory operation completes its execution.

$$\forall i, 1 \leq i \leq n, \forall k, 1 \leq k \leq \mathcal{J}_i, \forall x \in \mathcal{M}_{op}^i \setminus \{0, 1\}, (ST_{i,k,x} \geq ST_{i,k,x-2} + C_{i,k,x-2}).$$

For example, in Figure 6, the lower bound of the start time of the operation (1, 0, 3) is 2, as it is calculated from the start time of the previous memory operation (1, 0, 1) and its execution time (01 unit).

C4. Lower Bound of Start Time of Compute Operations: A compute operation (i, k, x) can start only when its previous two operations, i.e., the compute operation (i, k, x-2) and the memory operation (i, k, x-1), finish their execution.

$$\forall i, 1 \leq i \leq n, \forall k, 1 \leq k \leq \mathcal{J}_i, \forall x \in \mathcal{C}_{op}^i, ST_{i,k,x} \geq \max(ST_{i,k,x-1} + C_{i,k,x-1}, ST_{i,k,x-2} + C_{i,k,x-2}).$$

For example, in Figure 6, the start time of the compute operation (1, 0, 4) is calculated as the maximum of the start plus execution time of its previous compute operation (1, 0, 2) and memory operation (1, 0, 3).

C5. Finish Time of All Operations: The finish time can be derived by adding the execution time with the start time. Hence,

$$\forall i, k, 1 \leq i \leq n, 1 \leq k \leq \mathcal{J}_i, \forall x, 0 \leq x \leq p_i - 1, FT_{i,k,x} = ST_{i,k,x} + C_{i,k,x}.$$

C6. Upper Bound of Finish Time: To meet the deadline, a task τ_i should finish its execution before D_i . Since τ_i has p_i operations, the finish time of all its operations can be bounded as: $FT_{i,k,p_i-1} \leq (k+1) * D_i$, $FT_{i,k,p_i-2} \leq (k+1) * D_i - C_{i,k,p_i-1}$, and so on, where $(k+1) * D_i$ is the absolute deadline of k th job $J_{i,k}$ of τ_i . Hence,

$$\forall i, 1 \leq i \leq n, \forall k, 1 \leq k \leq \mathcal{J}_i, \forall x, 0 \leq x \leq p_i - 1, FT_{i,k,x} \leq (k+1) * D_i - \sum_{r=x+1}^{p_i-1} C_{i,k,r}.$$

For example, in Figure 6, the upper bound on the finish time of the last operation (1, 0, 5) is 7, since the absolute deadline of the job is 7 and the last operation has to finish within that deadline.

C7. Inter-job Start and Finish Time: A job $J_{i,k}$ of τ_i can only start execution when its previous job $J_{i,k-1}$ gets over. Hence,

$$\forall i, 1 \leq i \leq n, \forall k, 0 < k \leq \mathcal{J}_i, ST_{i,k,0} \geq FT_{i,k-1,p_i-1}.$$

Note that $ST_{i,k,0}$ is the start time of the *first* operation of $J_{i,k}$, while $FT_{i,k-1,p_i-1}$ is the finish time of the last operation of $J_{i,k-1}$.

C8. Stream Mapping Scenarios: A compute operation can be mapped to either a high- or low-priority stream at a time.

$$\forall i, k, 1 \leq i \leq n, 1 \leq k \leq \mathcal{J}_i, \forall x \in \mathcal{C}_{op}^i, Int(b_{1,i,k,x}) + Int(b_{2,i,k,x}) = 1.$$

Here, the function $Int(b)$ gives an integer encoding of the Boolean variable b . If $b = true/false$, then $Int(b) = 1/0$.

C9. Overlapping Scenarios: A pair of *different priority bound* compute operations (i_1, k_1, x_1) and (i_2, k_2, x_2) is said to be overlapping if they follow any of the four scenarios depicted in Figure 7. These scenarios are sketched based on the variations of the start and finish time of (i_1, k_1, x_1) and (i_2, k_2, x_2) ; e.g., scenario (a) reflects the situation when (i_1, k_1, x_1) subsumes (i_2, k_2, x_2) , and it can be formally modeled as $C_a : [(ST_{i_1,k_1,x_1} \leq ST_{i_2,k_2,x_2}) \wedge (FT_{i_2,k_2,x_2} < FT_{i_1,k_1,x_1})] \Rightarrow O_{(i_1,k_1,x_1)(i_2,k_2,x_2)}$. In this way, we model the other three scenarios and enable the decision variables for overlapping operations (ref. Section 3.3, V4). The final consolidated constraint is then,

$$\forall (i_1, k_1, x_1), (i_2, k_2, x_2), i_1 \neq i_2, 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, x_j \in \mathcal{C}_{op}^{i_j}, j = 1, 2, C_a \vee C_b \vee C_c \vee C_d.$$

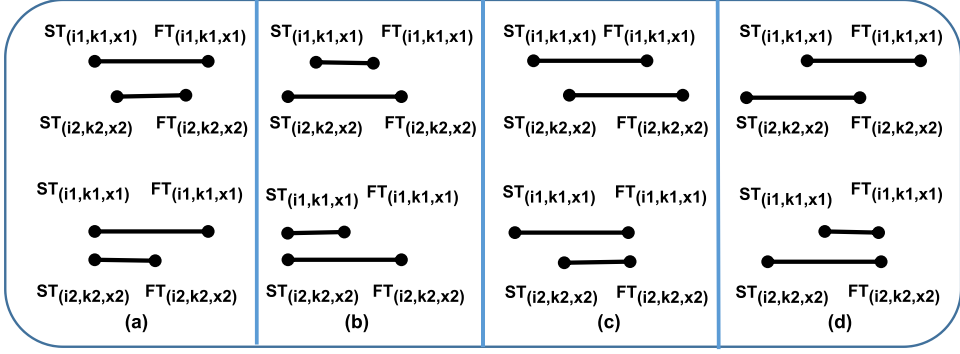


Fig. 7. Overlapping scenarios for compute operations.

C10. Non-overlapping Memory Operations: Since two memory operations cannot overlap in the same stream, we constrain the start and finish times of them as follows: If one operation starts earlier than the other, then it should finish earlier before the other starts.

$$\begin{aligned} & \forall (i_1, k_1, x_1), (i_2, k_2, x_2), i_1 \neq i_2, 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, \forall x_j \in \mathcal{M}_{op}^{i_j}, j = 1, 2, \\ & [(ST_{i_1, k_1, x_1} \leq ST_{i_2, k_2, x_2}) \Rightarrow \neg(ST_{i_2, k_2, x_2} < FT_{i_1, k_1, x_1})] \\ & \vee [(ST_{i_1, k_1, x_1} \geq ST_{i_2, k_2, x_2}) \Rightarrow \neg(ST_{i_1, k_1, x_1} < FT_{i_2, k_2, x_2})]. \end{aligned}$$

Note, here $i_1 \neq i_2$ enforces this constraint across tasks. For the same task, all of its operations follow their precedence rules as captured using the constraints C1 to C7.

C11. Non-overlapping Cases of Compute Operations: Two compute operations mapped to the same (either high or low) priority stream can not be overlapped. Hence,

$$\begin{aligned} & \forall (i_1, k_1, x_1), (i_2, k_2, x_2), i_1 \neq i_2, 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, x_j \in \mathcal{C}_{op}^{i_j}, j = 1, 2, \\ & [(b1_{i_1, k_1, x_1} \wedge b1_{i_2, k_2, x_2}) \vee (b2_{i_1, k_1, x_1} \wedge b2_{i_2, k_2, x_2})] \\ & \Rightarrow [(ST_{i_1, k_1, x_1} \leq ST_{i_2, k_2, x_2}) \Rightarrow \neg(ST_{i_2, k_2, x_2} < FT_{i_1, k_1, x_1})] \\ & \vee [(ST_{i_1, k_1, x_1} \geq ST_{i_2, k_2, x_2}) \Rightarrow \neg(ST_{i_1, k_1, x_1} < FT_{i_2, k_2, x_2})]. \end{aligned}$$

Recall, $b1_{i_1, k_1, x_1}$ and $b2_{i_1, k_1, x_1}$ are defined in Section 3.3.

C12. Immediately Preceding Memory Operation: (i_1, k_1, x_1) will be the immediately preceding memory operation of the current operation (i_2, k_2, x_2) if either (i_2, k_2, x_2) starts exactly when (i_1, k_1, x_1) finishes or there does not exist any other memory operation (i_3, k_3, x_3) executing in between (i_1, k_1, x_1) and (i_2, k_2, x_2) .

$$\begin{aligned} & \forall (i_1, k_1, x_1), (i_2, k_2, x_2), \nexists (i_3, k_3, x_3), 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, x_j \in \mathcal{M}_{op}^{i_j}, j = 1, 2, 3, \\ & (ST_{i_2, k_2, x_2} = FT_{i_1, k_1, x_1}) \vee ((ST_{i_3, k_3, x_3} \geq FT_{i_1, k_1, x_1}) \wedge (FT_{i_3, k_3, x_3} \leq ST_{i_2, k_2, x_2})) \Rightarrow IP_{M_{(i_1, k_1, x_1)(i_2, k_2, x_2)}}. \end{aligned}$$

As depicted in Figure 6, the immediately preceding memory operation of $(1, 1, 0)$ is $(0, 0, 3)$, hence, $IP_{M_{(0, 0, 3)(1, 1, 0)}}$ is set to *true*.

C13. Immediately Preceding Compute Operation: Using the same idea as stated in C12 for memory operation, we can find the immediately preceding compute operation of an operation mapped to high/low-priority stream.

$$\begin{aligned} & \forall (i_1, k_1, x_1), (i_2, k_2, x_2), \nexists (i_3, k_3, x_3), 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, x_j \in \mathcal{C}_{op}^{i_j}, j = 1, 2, 3, \\ & [(b1_{i_1, k_1, x_1} \wedge b1_{i_2, k_2, x_2} \wedge b1_{i_3, k_3, x_3}) \wedge (ST_{i_2, k_2, x_2} = FT_{i_1, k_1, x_1})] \\ & \vee (ST_{i_3, k_3, x_3} \geq FT_{i_1, k_1, x_1} \wedge FT_{i_3, k_3, x_3} \leq ST_{i_2, k_2, x_2}) \Rightarrow IP_{H_{(i_1, k_1, x_1)(i_2, k_2, x_2)}} \end{aligned}$$

$$\bigwedge [(b2_{i_1, k_1, x_1} \wedge b2_{i_2, k_2, x_2} \wedge b2_{i_3, k_3, x_3}) \wedge (ST_{i_2, k_2, x_2} = FT_{i_1, k_1, x_1}) \\ \vee (ST_{i_3, k_3, x_3} \geq FT_{i_1, k_1, x_1} \wedge FT_{i_3, k_3, x_3} \leq ST_{i_2, k_2, x_2}) \Rightarrow IP_{L_{(i_1, k_1, x_1)(i_2, k_2, x_2)}}].$$

Note that the constraint written in boldface is for high-priority stream, and the regular one is for low-priority stream. For example, according to Figure 6, the immediately preceding compute operation of the high-priority bound compute operation (0, 0, 4) is (1, 0, 2), and hence, $IP_{H(1, 0, 2)(0, 0, 4)}$ is set to *true*. Also, $IP_{L(1, 0, 4)(0, 0, 2)}$ is *true*, since (1, 0, 4) immediately precedes the compute operation (0, 0, 2) in the low-priority stream.

C14. Last Overlapping High-priority Operation: Consider compute operations (i_1, k_1, x_1) and (i_2, k_2, x_2) mapped to the high- and low-priority streams, respectively. Now, (i_1, k_1, x_1) is the last operation to overlap with (i_2, k_2, x_2) if the following three conditions hold together: (1) (i_1, k_1, x_1) and (i_2, k_2, x_2) overlap, (2) (i_1, k_1, x_1) finishes earlier than (i_2, k_2, x_2) , (3) there does not exist any other compute operation (i_3, k_3, x_3) mapped to high-priority stream that overlaps with (i_2, k_2, x_2) and starts after (i_1, k_1, x_1) finishes.

$$\forall (i_1, k_1, x_1), (i_2, k_2, x_2), \nexists (i_3, k_3, x_3), 1 \leq i_j \leq n, 1 \leq k_j \leq \mathcal{J}_{i_j}, x_j \in \mathcal{C}_{op}^{i_j}, j = 1, 2, 3, \\ [b1_{i_1, k_1, x_1} \wedge b2_{i_2, k_2, x_2} \wedge O_{(i_1, k_1, x_1)(i_2, k_2, x_2)} \wedge (FT_{i_1, k_1, x_1} \leq FT_{i_2, k_2, x_2}) \wedge \\ (b1_{i_3, k_3, x_3} \wedge O_{(i_2, k_2, x_2)(i_3, k_3, x_3)} (FT_{i_1, k_1, x_1} < ST_{i_3, k_3, x_3}))] \Rightarrow LO_{H_{(i_1, k_1, x_1)(i_2, k_2, x_2)}}.$$

According to the example in Figure 6, the last overlapping operation for the low-priority bound operation (1, 1, 2) is (0, 0, 4), as it satisfies all the three conditions specified earlier.

C15. Start Time of Memory Operation: The start time of a memory operation (i, k, x) is the maximum of its arrival time and the finish time of its immediately preceding operation, (i', k', x') .

$$\forall (i, k, x), \exists (i', k', x'), 1 \leq i, i' \leq n, 1 \leq k \leq \mathcal{J}_i, 1 \leq k' \leq \mathcal{J}_{i'}, x \in \mathcal{M}_{op}, x' \in \mathcal{M}_{op}^{i'}, IP_{M_{(i', k', x')(i, k, x)}} \wedge ST_{i, k, x} = \max(a_{i, k, x}, FT_{i', k', x'}).$$

For example, the start time of the memory operation (1, 1, 0) as specified in Figure 6 can be calculated as: $ST_{1, 1, 0} = \max(a_{1, 1, 0}, FT_{0, 0, 3})$, since $IP_{M(0, 0, 3)(1, 1, 0)} = \text{true}$, i.e., (0, 0, 3) is the immediately preceding memory operation of (1, 1, 0).

C16. Start Time of High-priority Compute Operation: Using similar reasoning as stated in C15, the start time of a compute operation mapped to the high-priority stream can be derived.

$$\forall (i, k, x), \exists (i', k', x'), 1 \leq i, i' \leq n, 1 \leq k \leq \mathcal{J}_i, 1 \leq k' \leq \mathcal{J}_{i'}, x \in \mathcal{C}_{op}^{i'}, x' \in \mathcal{C}_{op}, IP_{H_{(i', k', x')(i, k, x)}} \wedge ST_{i, k, x} = \max(a_{i, k, x}, FT_{i', k', x'}).$$

The expression for the start time of the compute operation (0, 0, 4) bound to the high-priority stream as depicted in Figure 6 is $ST_{0, 0, 4} = \max(a_{0, 0, 4}, FT_{1, 0, 2})$, since $IP_{H(1, 0, 2)(0, 0, 4)} = \text{true}$.

C17. Start Time of Low-priority Compute Operation: The start time of a compute operation (i, k, x) mapped to the low-priority stream can be calculated following two factors: (1) When there is no overlapping operation in the high-priority stream; the start time will be the maximum of its arrival time and the finish time of its immediately preceding operation (i', k', x') in the low-priority stream. (2) When there exist overlapping operations in the high-priority stream that are not co-schedulable with (i, k, x) ; the start time will be the maximum of its arrival time and finish time of the last overlapping operation (i'', k'', x'') in the high-priority stream.

$$\forall (i, k, x), \exists [(i', k', x') \vee (i'', k'', x'')], 1 \leq i, i', i'' \leq n, 1 \leq k \leq \mathcal{J}_i, \\ 1 \leq k' \leq \mathcal{J}_{i'}, 1 \leq k'' \leq \mathcal{J}_{i''}, x \in \mathcal{C}_{op}^i, x' \in \mathcal{C}_{op}^{i'}, x'' \in \mathcal{C}_{op}^{i''}, \\ b2_{i, k, x} \wedge [IP_{L_{(i', k', x')(i, k, x)}} \vee (LO_{H_{(i'', k'', x'')(i, k, x)}} \wedge \neg S_{(i'', k'', x'')(i, k, x)})] \wedge \\ [ST_{i, k, x} = \max(a_{i, k, x}, FT_{i', k', x'}, FT_{i'', k'', x''})].$$

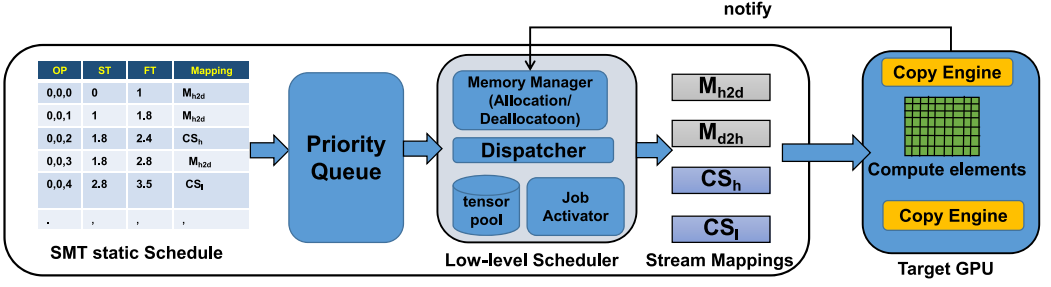


Fig. 8. Low-level SMT scheduler.

Recall $S_{(i'', k'', x'')(i, k, x)}$ has been defined in Section 3.3. As outlined in Figure 6, for the low-priority bound compute operation $(1, 1, 2)$, its immediately preceding compute operation in the low-priority stream is $(0, 0, 2)$ (i.e., $IP_{L(0,0,2)(1,1,2)} = true$) and last overlapping high-priority operation is $(0, 0, 4)$ (i.e., $LO_{H(0,0,4)(1,1,2)} = true$). Also, for this example, it is assumed that the operations $(1, 1, 2)$ and $(0, 0, 4)$ can be co-scheduled together (i.e., $S_{(0,0,4)(1,1,2)} = true$). Then the expression for the start time of compute operation $(1, 1, 2)$ is: $ST_{1,1,2} = \max(a_{1,1,2}, FT_{0,0,2})$.

C18. The Objective Function: Since our goal is to minimize the weighted sum of WCRTs of all the jobs of all the tasks, we formulate the objective function as,

$$\text{Min} \sum_{i,k} w_i \cdot (FT_{i,k,p_{i-1}} - a_{i,k,0}) \text{ with } 1 \leq i \leq n, 1 \leq k \leq \mathcal{J}_i, w_i = \frac{1}{h_i},$$

subject to the consolidated constraint: $C1 \wedge C2 \wedge \dots \wedge C16 \wedge C17$.

The weight factor chosen for our problem formulation helps the SMT to schedule tasks based on their periods, i.e., the lower the value of period (h), the higher the weight assigned to the jobs of that task; hence, SMT will try to schedule it before other jobs with larger periods. In other words, tasks with lower period values get higher weight, thus indirectly giving more preference for completion and eventually leading to an optimal schedule with minimized WCRT of all tasks. Note, our constraint and objective function formulation can account for any arbitrary weight factors for different tasks. For example, in mixed-criticality scheduling, where tasks' criticality is more important than their periods [3], it makes sense to choose weight factors that capture the relative criticality levels of tasks.

In addition to the above SMT formulation, we have implemented a low-level scheduler that executes as a service on the target GPGPU platform, processing any given static schedule represented in the form of an operation-to-stream mapping table.

4 LOW-LEVEL SCHEDULER

Our proposed low-level scheduler is essentially a host side process acting as a dispatcher, dispatching operations on the target device according to the start time and stream mapping information. Figure 8 depicts a diagrammatic representation of the low-level scheduler.

The input to the low-level scheduler is the operation-to-stream mapping table, which captures information relevant to each operation. This includes operation ID, start time, finish time, and the stream to which an operation is mapped. The low-level scheduler maintains a priority queue where operations are stored in ascending order of their start times. It also maintains a fixed set of CUDA memory and compute streams with different priority levels (as discussed earlier). The job activator takes care of ensuring when jobs of a task should be initiated. This is done by maintaining a running timer using high-precision clocks supported in the standard C++ *chrono* library. Depending on when jobs become ready for execution, i.e., the start times equal to timestamps reported

by the running timer, the low-level scheduler dequeues its constituent operations one-by-one and dispatches them to streams specified in the mapping table. During dispatch, the memory manager module carries out necessary tensor allocations. Once the operation completes its execution on the GPU, a notify signal is sent back to the low-level scheduler. The timestamp at which the notify signal was spawned is compared with that of the running timer to determine if operations have met their deadlines or not. Once an operation is finished, the associated tensors are subsequently deallocated.

Since the primary objective of the SMT-optimizer is to minimize worst-case response time, we observe that the SMT directs required tensors to be greedily prefetched to memory as and when the designated memory streams are available, irrespective of when their associated compute operations start. This is because the SMT constraints in their current form dictate that jobs with earlier deadline requirements would be prioritized and accordingly scheduled to high-priority compute streams. The formulation is oblivious to when memory operations can be suitably scheduled. As a result of this aggressive prefetching of tensors, a considerable amount of memory allocation is done at the beginning, resulting in high memory occupancy. Allocated memory is freed when the corresponding compute operations in the schedule get executed. We next discuss techniques on how this can be mitigated.

5 IMPROVING MEMORY CONSUMPTION

One way to solve the above-mentioned aggressive memory prefetching issue is to model suitable constraints in the current SMT formulation and generate a schedule balancing the memory consumption. Towards this, we need to model constraints by considering overlapping operations and then calculating the maximum memory requirement in those overlapping intervals, followed by checking whether the requirement satisfies the given memory budget or not. However, this approach would lead to an exponential increase in the number of constraints, as outlined below.

Consider a system setting having total k pipelines where i th pipeline has m_i number of operations. Therefore, there are approximately $\frac{m_i}{2}$ memory and $\frac{m_i}{2}$ compute operations, since compute and memory operations alternate in the task pipeline. To formulate constraints for memory consumption, we need to keep track of total memory utilization during the execution of each operation. Now, the total memory utilization during an operation is the maximum memory required among all other overlapping operations of the operation in consideration. To find out all such overlapping operations, SMT has to check $m_1 \times m_2 \times \dots \times m_k$ combinations of operation pairs in the worst-case scenario. A combination of such pairs of operations becomes significantly high for pipelines with moderate size. Moreover, modeling the constraint for finding the maximum among several such numbers is also non-linear. Hence, the process of mitigating aggressive memory prefetching by exploiting SMT formulation is compute-intensive and not implementable in practice.

To this end, we envision an offline heuristic performing better than the SMT-based approach, as mentioned above. We propose a two-pass heuristic method that takes the SMT-reported static schedule as input and performs relevant shifting operations without violating the memory budget requirement and deadline requirement of each job. In particular, in the first pass, memory operations are rearranged and shifted towards the right (across the hyper-period) by a suitable phase shift amount that puts some delay in bringing weight tensors occupying a bulk amount of memory. In the second pass, compute operations and their corresponding memory operations are tested for possible shifting towards the right if there is no memory budget violation and free slots are available in respective streams. This overall approach reduces the peak memory consumption during the scheduling and tries to flatten it by spreading memory-consuming operations across the hyper-period. The detail of the proposed method, entitled **Peak Memory Shaving (PMS)** algorithm, is discussed next.

ALGORITHM 1: PMS Pass-I**Input:** Static SMT schedule $\mathcal{S}$, memory budget mb **Output:** Updated SMT schedule $\mathcal{S}$

```

1  $list_{CS_h}, list_{CS_l}, list_{h2d}, list_{d2h} \leftarrow preprocess(\mathcal{S});$ 
2  $free\_slots = initialize\_free\_slots(list_{h2d});$ 
3 while  $not\_empty(list_{CS_h}, list_{CS_l})$  do
4    $cop \leftarrow get\_latest\_compute\_op(list_{CS_h}, list_{CS_l});$ 
5    $mop \leftarrow get\_memory\_op(cop, list_{h2d});$ 
6    $fs \leftarrow matching\_free\_slots(mop.FT, cop.ST, free\_slots);$ 
7    $shift\_op(mop, fs, mb, list_{h2d}, \mathcal{S}, free\_slots)$ 
8 return  $\mathcal{S};$ 

```

5.1 Pass-I of PMS Algorithm

We first discuss the first pass of PMS method with the help of Algorithm 1. The purpose of the first pass is to spread only memory operations across the hyper-period and reduce the peak memory consumption due to aggressive tensors' prefetching. The algorithm takes the static SMT schedule $\mathcal{S}$ and returns an updated schedule with modified start and finish timings for memory operations. Using the *preprocess()* routine, individual lists of operations mapped to each of the four streams $\{M_{h2d}, M_{d2h}, CS_h, CS_l\}$ are first obtained (line 1). The lists $list_{CS_h}$ and $list_{CS_l}$ contain operations mapped to high-priority and low-priority streams, respectively. The lists $list_{h2d}$ and $list_{d2h}$ contain memory operations for H2D and D2H transfers, respectively. The constituent operations of each of the four lists are sorted in ascending order of their start times. Let us denote a free interval slot by the tuple (s_{slot}, f_{slot}) , where s_{slot} and f_{slot} represent the start and finish times of that slot. Depending on the operations' start and finish times, multiple free slots can exist in any given stream. Using the *initialize_free_slots()* routine, we obtain a list of free slots from $list_{h2d}$ and store in $free_slots$ (line 2). The algorithm next processes each compute operation (maintained in the lists $list_{CS_h}$ and $list_{CS_l}$) in descending order of their start times, identifies its relevant H2D operation, and tries to shift to available free slots to reduce the overall memory consumption (lines 3–8). In each iteration of the while loop (line 3), the compute operation cop with the highest start time is first removed from either $list_{CS_h}$ or $list_{CS_l}$ using *get_latest_compute_op()* (line 4). The corresponding memory operation mop for cop is identified from $list_{h2d}$ using *get_memory_op()* routine (line 5). Next, the *matching_free_slots()* routine is used to extract the set of free slots fs from $free_slots$ where the start and finish times of each $slot \in fs$ reside between the finish time of mop and start time of cop . The set fs represents available time intervals, where mop can be appropriately shifted without violating the timing requirement of cop . Once the set fs is obtained for mop , we call procedure *shift_op()* to shift the mop in suitable free slots. The pseudo-code for *shift_op()* is described in Algorithm 2 and discussed next.

We pass the memory operation mop , matching free slots fs , memory budget mb , H2D operation list $list_{h2d}$, current schedule $\mathcal{S}$, and list of $free_slots$ as input parameters to Algorithm 2. In the *for* loop of Algorithm 2, each $slot \in fs$ is processed in descending order of its finish times (lines 1–7). Each iteration checks whether the operation op can be shifted to the free slot $slot$. The operation can be shifted *iff* no memory budget violation is encountered due to this shifting. The routine *memory_budget_violation()* (line 2) checks if the memory footprint of the schedule after shifting is greater than the supplied memory budget mb or not. If there is no memory budget violation, then the amount of shift required *phase_shift* is calculated by the routine *obtain_phase_shift()* (line 3). The operation is shifted to the right by *right_shift()* routine (line 4) and the schedule $\mathcal{S}$ along with $free_slots$ are updated accordingly (lines 5 and 6). This process of shifting each

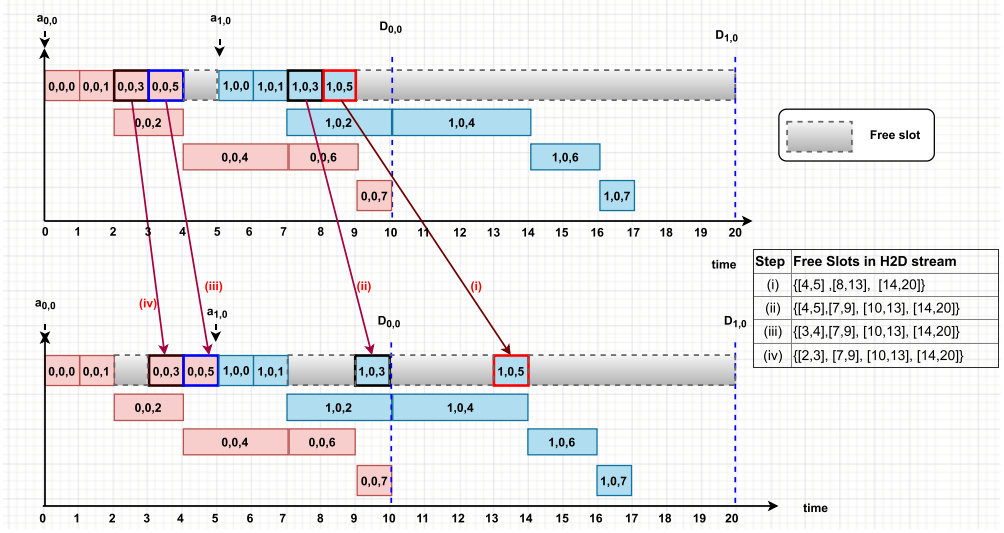


Fig. 9. Example execution of Pass-I algorithm.

ALGORITHM 2: Procedure to shift operation

Input: Operation op , matching free slots fs , memory budget mb , list of operations $list$, Schedule S , list of free slots $free_slots$

Output: updated Schedule S , updated $free_slot$ list

Procedure $Shift_Op(op, fs, mb, list, S, free_slots)$

```

1 for slot  $\in fs$  in decreasing order of finish time do
2   if  $memory\_budget\_violation(op, slot, mb) == false$  then
3      $phase\_shift = obtain\_phase\_shift(op, slot);$ 
4      $right\_shift(op, phase\_shift, list);$ 
5      $update(S);$ 
6      $update(free\_slots);$ 
7 end Procedure

```

relevant memory operation for every compute operation is continued as long as no violation in terms of memory budget is encountered.

We consider a schedule consisting of two tasks τ_0 and τ_1 , each comprising eight operations, and illustrate a running example of Algorithm 1 with the help of Figure 9. The arrival times of τ_0 and τ_1 are 0 and 5, respectively, and deadlines are 10 and 20, respectively. The top part of Figure 9 represents the Gantt chart of the schedule reported by the SMT-optimizer, whereas the bottom part shows the modified Gantt chart after applying Algorithm 1. The light-orange slots represent the time intervals of the operations for task τ_0 , and light-blue slots are for task τ_1 . In the figure, free slots in the memory streams are marked by grey color. The table on the right side of the figure lists the status of the $free_slots$ variable in the algorithm during each step. The red arrow depicts the corresponding movements of the memory operation during that step. As an example, during the first step, the latest compute operation cop in the given schedule is (1, 0, 6), and the corresponding memory operation mop is (1, 0, 5). The red arrow marked with (i) depicts the movement of the memory operation (1, 0, 5) from time slot [8 – 9] to free slot [13 – 14] assuming there is no memory budget violation. It can be seen from the schedule drawn in the bottom part of Figure 9 that

the memory operations for τ_0 and τ_1 are spread across the timeline, thereby reducing the overall memory consumption at any point of time during the schedule.

5.2 Pass-II of PMS Algorithm

As illustrated in the previous section, the target of the first pass of PMS algorithm is to restructure the sequence of memory operations so it happens closer to their respective compute operations. On the other side, the primary aim of the second pass of the PMS method is to relax compute operations, i.e., sometimes allow them to finish beyond their deadlines but within a tolerable threshold. In a practical use case like vehicular **Automated Driver Assisted Systems (ADAS)**, we have multiple perception tasks, such as pedestrian detection, lane detection, traffic sign recognition, and so on, running simultaneously. In such a system, the associated hard real-time critical control tasks (e.g., autonomous emergency braking) are often designed pessimistically with sampling frequencies set much higher than safety margins [6]. Such demanding execution is often possible, because the task itself may be lightweight in nature. However, the associated perception systems are quite heavy to execute and even if a few of the inferences miss their deadlines, the delayed perception feedback can be considered admissible and used by the control task in the next sampling period [6]. This notion of limited relaxation of deadlines would potentially allow more scope for optimizing memory utilization of tasks at runtime. To emulate this idea, we consider the list d_relax containing the deadline relaxation parameter for each task. The deadline requirement D_i for the i th task becomes $D + d_relax[i] \times D$ due to this relaxation. The algorithm for pass-II shifts compute operations, satisfying the relaxed deadlines of the tasks across the hyper-period. Along with the compute operations, corresponding memory operations are also getting shifted while ensuring that the memory footprint remains under the memory budget. Algorithm 3 describes this process in detail.

ALGORITHM 3: PMS Pass-II

Input: Schedule $\mathcal{S}$ produced by PMS Pass-I, list of deadline relaxation percentage d_relax , memory budget mb

Output: Updated schedule $\mathcal{S}$

```

1  $list_{CS_h}, list_{CS_l}, list_{h2d}, list_{d2h} \leftarrow preprocess(\mathcal{S});$ 
2  $free\_slots_{CS_h}, free\_slots_{CS_l}, free\_slots_{h2d} = initialize\_free\_slots(list_{CS_h}, list_{CS_l}, list_{h2d});$ 
3  $shift\_d2h\_operations(list_{d2h}, d\_relax);$ 
4 while  $not\_empty(list_{CS_h}, list_{CS_l})$  do
5    $stream, cop \leftarrow get\_latest\_compute\_op(list_{CS_h}, list_{CS_l});$ 
6    $nop \leftarrow get\_next\_op(cop);$ 
7    $fs \leftarrow matching\_free\_slots(cop.FT, nop.ST, free\_slots_{stream});$ 
8    $shift\_op(cop, fs, mb, list_{stream}, \mathcal{S}, free\_slots);$ 
9    $mop \leftarrow get\_memory\_op(cop);$ 
10   $fs \leftarrow matching\_free\_slots(mop.FT, cop.ST, free\_slots_{list_{h2d}});$ 
11   $shift\_op(mop, fs, mb, list_{h2d}, \mathcal{S}, free\_slots)$ 
12 return  $\mathcal{S};$ 
```

Algorithm 3 takes the schedule $\mathcal{S}$ produced by the Algorithm 1 as input along with the deadline relaxation factor d_relax and the memory budget mb . Similar to Algorithm 1, Algorithm 3 also uses the $preprocess()$ routine (line 1) to get the four lists $list_{CS_h}$, $list_{CS_l}$, $list_{d2h}$, and $list_{h2d}$ containing list of operations mapped to the four streams. Similar to pass-I, the operations in each list are sorted in descending order of their start time values. In contrast to Algorithm 1, the $initialize_free_slots()$ routine is used in Algorithm 3, which takes $list_{CS_h}$, $list_{CS_l}$, and $list_{h2d}$ as input and returns three

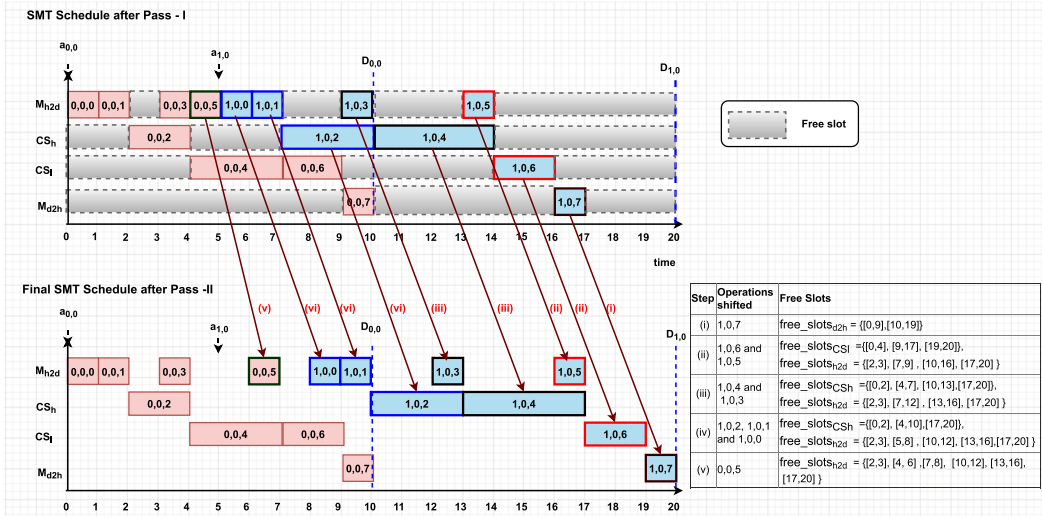


Fig. 10. Example execution of Pass-II algorithm.

lists, $free_slots_{CS_h}$, $free_slots_{CS_l}$, and $free_slots_{h2d}$, of free slots (line 2). The lists $free_slots_{CS_h}$ and $free_slots_{CS_l}$ represent slots available in the high (CS_h) and the low-priority (CS_l) compute streams, respectively. The list $free_slots_{h2d}$ represents free time slots in the memory stream for H2D transfers. In the next step, the start time of every job's last D2H operation is readjusted as per the relaxation parameters given in d_relax . This is done by invoking the `shift_d2h_operations()` routine (line 3). The objective of pass-II is to process compute operations in the descending order of their start times and subsequently identify their associated H2D memory operations. Each pair of compute and H2D operations is subsequently checked for possible shifting while maintaining memory budget constraints (lines 4–11).

In each iteration of the while loop (line 4), the latest compute operation cop and its mapped stream $stream$ is obtained by calling the `get_latest_compute_op()` routine (line 5). Given the cop , its immediate successor operation in the pipeline is identified and stored in nop by calling the `get_next_op()` routine (line 6). We note that nop can either be a compute operation or the last D2H operation for any job. The intuition behind identifying nop is that any compute operation can not be shifted beyond the successive compute/D2H operation in the pipeline due to data dependency. The algorithm then checks for available free slots between the finish time of cop and the start time of nop and stores the slots in fs invoking the routine `matching_free_slots()` (line 7). Next, we call the `shift_op()` procedure (line 8) to check the possibility of shifting cop in the matching free slots fs while maintaining the memory budget mb . The preceding memory operation mop for the compute operation cop is obtained by calling the routine `get_memory_op()` (line 9) and free slots between the finish time of mop and the start time of cop are obtained by invoking `matching_free_slots()` routine (line 10). Next, we check for the possibility of shifting of memory operation mop by calling the procedure `shift_op()` (line 11). This process rearranges the compute and memory operations along the hyper-period so at any point in time total memory footprint of the scheduler is always less than the memory budget provided. During the execution of Algorithm 3, at any step, if the memory budget is violated either for compute operation cop or for memory operation mop , then the operation is not shifted to the free slot.

The execution of Algorithm 3 is illustrated in Figure 10 for the example of two tasks, as explained in the previous section. Column-1 of the table on the right side of the figure indicates the step ID,

operations that are shifted following Algorithm 3 are reported in Column-2, and the status of free slots in all the streams are reported in Column-3. The actual shifting of compute and dependent memory operations corresponding to the step IDs listed in the table are shown using red arrows in the figure. Consider the shifting of the compute operation (1, 0, 6) mapped to the low-priority compute stream (i.e., CS_l) and the corresponding memory operation (1, 0, 5). For (1, 0, 6), the next operation *nop* is the d2h operation (1, 0, 7). Note, there are free slots in the compute stream CS_l within the time interval [16 – 19]. This is because the d2h operation (1, 0, 7) has been shifted in the step-(i) of Algorithm 3 from the current slot of [16 – 17] to a free slot of [19 – 20], as the deadline of τ_1 is 20, and there is a free slot available within the time interval [17 – 20] in the M_{d2h} stream. Hence, the operation (1, 0, 6) is shifted to the right and moved to the time slot [17 – 19] assuming there is no memory budget violation. Accordingly, the dependent memory operation (1, 0, 5) is shifted to the new slot [16 – 17] from the current slot [13 – 14] just before the start of the compute operation (1, 0, 6), as the free slot was available and memory budget is not violated. Similarly, the remaining compute and memory operations are processed, and the final schedule is depicted in the bottom part of Figure 10.

Complexity analysis of PMS algorithm: The Pass-I algorithm runs for every compute operation mapped to CS_h and CS_l streams. Let us consider that for the given set of tasks, there are total m jobs, and each task has n operations. Since compute and memory operations alternate in the task pipeline, there will be at most $n/2$ compute and $n/2$ memory operations per job. Since the procedure *preprocess()* (line 1 of Algorithm 1) returns four lists of operations sorted as per the ascending order of their start times, it takes $O(n \log n)$ time. Next, the *while* loop in Algorithm 1 executes for at most $m \times (n/2)$ times and under each such iteration, a constant amount of time (i.e., $O(1)$) is needed to get *cop* and *mop*. On the other side, *matching_free_slots()* scans through the list *free_slots*. The maximum length of the free slot can be $m \times (n/2 + 1)$. Hence, the worst-case time complexity of *matching_free_slots()* is $O(mn)$. The time complexity of *shift_op()* is also $O(mn)$. This is because the *for* loop in *shift_op()* is iterated for the number of matching free slots *fs* in the memory stream. Now, for m jobs with $(n/2)$ memory operations per job, the maximum number of free slots in the memory stream is $m \times (n/2) + 1$. Therefore, the worst-case time complexity of Algorithm 1 can be asymptotically bounded by $m \times (n/2) \times [O(n \log n) + O(1) + O(mn) + O(mn)] = O(m^2 n^2)$. In Algorithm 3, i.e., Pass-II, *preprocess()* routine executes for $O(n \log n)$ and the routine *shift_d2h_operation()* executes for $O(m)$ time. The *while* loop runs for every compute operation, i.e., in the worst case it runs for $m \times (n/2 + 1)$ times. The routine *get_next_op()* finds the *nop* in $O(1)$ time and as per the discussion above, *matching_free_slots()* routine identifies the set of matching free slots with worst-case time complexity of $O(mn)$. Considering the time complexity of *shift_op()* is $O(mn)$ and *get_memory_op()* as $O(1)$ as discussed above, the worst-case time complexity of Algorithm 3 can be asymptotically bounded by $m \times (n/2) \times [O(n \log n) + O(m) + O(1) + O(1) + O(mn) + O(mn) + O(1) + O(mn) + O(mn)] = O(m^2 n^2)$.

We note that our proposed PMS algorithm is a heuristic that does not always guarantee that the memory overhead remains smaller than the memory budget supplied by the user. The proposed scheme tries to shift both memory and compute operations greedily whenever possible by suitably relaxing deadlines of tasks. Furthermore, we note that the memory budget can not be set to arbitrary values. The budget should at least be greater than the maximum memory requirement of weight and input-output tensors for a layer in the pipeline to ensure execution. Also, for ensuring concurrent execution and maximal exploitation of GPU resources, one must ensure that the budget supplied should at least support the accommodation of individual layer requirements for two different pipelines. Given these minimal requirements for the memory budget, if there exists sufficient space for shifting the operations, i.e., the deadlines can be sufficiently relaxed with respect

Table 1. Experiment Configuration

Exp. ID.	Hyper-period configuration	#Jobs per Hyper-Period	TI (1e6)	%-Gain SMT-LS vs. LSF-LS	%-Gain SMT-PMS-LS vs. LSF-LS
exp1	[60,30]	3	1.64	78.62	96.31
exp2	[20,40]	3	2.44	79.98	84.23
exp3	[30,30,30]	3	3.25	73.22	78.25
exp4	[20,60]	4	2.16	83.88	86.58
exp5	[30,60,60]	4	2.16	83.77	89.08
exp6	[20,80]	5	2.03	81.80	85.44
exp7	[20,30]	5	2.72	76.70	84.24
exp8	[30,60,30]	5	2.77	82.37	87.52
exp9	[30,30,60]	5	2.70	81.48	88.19
exp10	[20,100]	6	1.94	84.60	87.88
exp11	[20,120]	7	1.89	86.22	90.64
exp12	[30,40]	7	1.90	85.07	87.00
exp13	[40,30]	7	1.90	74.52	78.68
exp14	[20,140]	8	1.85	90.12	98.50

to the actual running times of constituent tasks, then our proposed scheme can suitably delay compute and memory-bound operations to reduce the memory overhead within the prescribed budget.

6 EXPERIMENTS AND RESULTS

We evaluate the efficacy of our approach by running multiple scheduling experiments involving the two well-known pipelines, YoloLite [18] and Tiny-Yolov2-VOC [42], which have been extensively used in the embedded edge domain to perform object detection tasks. The aforementioned pipelines represent standard choices for object detection tasks, owing to their robustness for localizing and classifying objects in images [11, 16, 37, 45, 46, 55, 56]. We also note that such pipelines have been primary candidates for experiments in other popular real-time scheduling works as well [52, 53, 57]. For evaluating our use case, scheduling a combination of these pipelines with each pipeline trained and deployed for a specific object detection task serves as a good practical scenario.

Experiment Setting: As the target platform, we have considered the NVIDIA Jetson Xavier NX embedded board, which is equipped with (i) an NVIDIA Volta GPU and (ii) an NVIDIA Carmel ARMv8.2 CPU. We consider a total of 14 experiments, out of which 10 are composed of two pipelines, i.e., YoloLite and tiny-Yolov2-VOC, and the remaining 4 have three pipelines, i.e., YoloLite, tiny-Yolov2-VOC, and YoloLite. The details for each such experiment are depicted in Table 1. For each experiment ID listed in column 1 of Table 1, we have the hyper-period configuration, and the total number of jobs for the current configuration is listed in columns 2 and 3, respectively. Column 4 lists the **Throughput Index (TI)** values [14] (in the order of $1e6$) for each experiment configuration. Here, TI represents a measure of throughput requirement for any given experiment configuration and is calculated as $TI(\tau_i) = \sum_i FLOPS(\tau_i)/h_i$, where $FLOPS(\tau_i)$ is the total number of floating point operations required for each pipeline τ_i , and h_i is its corresponding period. Typically, configurations with higher (lower) period values indicate a lower (higher) throughput requirement. For each experiment configuration, we run scheduling schemes that leverage the low-level scheduler, which operates on the static schedules produced by the different algorithms discussed so far. These include (i) baseline **least slack first scheduling scheme (LSF-LS)** (ii) **SMT static schedule executed using the low-level scheduler (SMT-LS)**, and (iii) **SMT static schedule processed by PMS and finally executed by low-level scheduler (SMT-PMS-LS)**. For producing the SMT static schedules, we use the SMT-Optimizer Z3 [33] in our experiments.

We ran our SMT optimizer experiments on a server machine with 24-core Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20 GHz with 64 GB RAM. We observed that our experiment configuration with

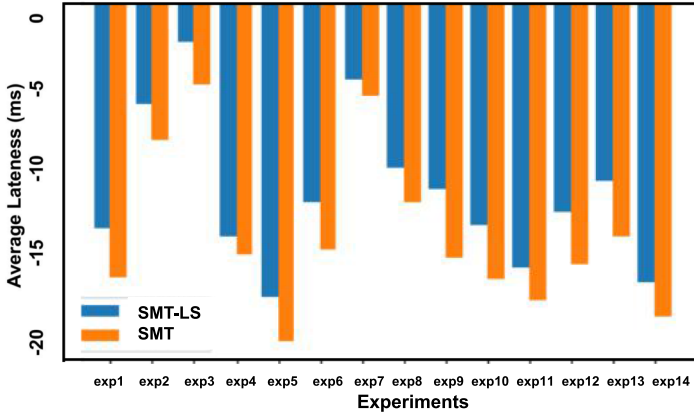


Fig. 11. Avg. lateness: SMT-LS vs. SMT.

two tasks took time in the range of 0.794 – 232.446 hours, while our configuration with three tasks took time in the range of 8.2338 – 221.2544 hours to produce an optimized schedule. This happens because of the extremely complex space of choices and constraints, as discussed earlier. We observed, specifically for experiments with IDs 8, 9, and 12, that the SMT optimizer took the largest amount of time. These experiments had a relatively higher number of jobs, in contrast to other experiments, but more importantly, they had an equal distribution of jobs between the tasks considered in the respective experimental task sets. In these cases, the high execution times for our SMT-solver may be attributed to the potential number of choices arising from scheduling operations for jobs of different tasks concurrently on the target GPU. Our robust SMT design ensures termination and reasonable memory consumption (a maximum of 26 GB, i.e., 40% of the memory resources of the target platform considered) for all experiments considered. This is admissible given that the entire process of obtaining schedules is offline. The static schedules obtained using the SMT optimizer are deployed to the low-level scheduler.

We first investigate the efficacy of the schedules finally produced by our low-level scheduler by performing a comparative evaluation between the schedule produced by SMT-LS and the static schedule produced by only the SMT optimizer. In particular, we investigate the average lateness experienced by jobs in both schedules for all 14 experiments. A bar graph indicating the average lateness values between the two schedules is illustrated in Figure 11. The blue-and-orange-colored bar represents the average lateness of the schedule produced by SMT-LS and SMT, respectively. One can observe that for both schedules produced by SMT-LS and only SMT, the average lateness is negative. Furthermore, the average lateness values for the schedules produced by only SMT are lesser than that of SMT-LS. This may be attributed to the low-level scheduler incurring additional overheads arising from stream management, resource contention, and so on, which are not modeled in our SMT formulation. This causes some jobs to finish later than their finishing times, as reported in the static schedule by the SMT optimizer. We observed that the percentage of increase in average lateness for schedules produced by only SMT-LS compared to that of only SMT schedule for our experiment configuration varies in the range of 8%–35%. While we observe large deviations in terms of average lateness between SMT-LS and only SMT, this does not reflect the lateness experienced per job in any given schedule. We computed the percentage change in lateness values w.r.t. deadlines of each job for each experiment. We observed that for any given job, the increase in lateness as a result of running SMT-LS ranges from 1.1% to 12.33% of its deadline, thus indicating that there is relatively less deviation in performance with respect to only SMT. Our next set of experiments evaluates the performance between SMT-LS and our baseline approach LSF-LS.

For comparison, we have also implemented the baseline LSF scheme introduced in Section 1 using our low-level scheduler. Unlike our proposed scheme, in LSF, the scheduler maintains several CUDA compute streams, each with the same priority level. The total number of streams equals the total number of scheduled tasks. Instead of using start times given by the SMT module, the kernel dispatching order in this context is decided by the slack time of the kernel. Slack time denotes the remaining time within which the kernel has to complete its execution so its deadline is respected. Slack value for each kernel is calculated as $D - t - ret$, where D is the job's deadline, t is the current time, and ret is the remaining execution time of the kernel. The kernel with the least slack time value is selected for dispatch. The inherent problem with this approach is that there is no priority differentiation between tasks. Furthermore, there is no scope for asynchronous overlapping of memory operations over compute operations, since it is assumed that tensors are resident in memory beforehand.

While scheduling, the number of streams used for the baseline approach depends on the number of periodic pipelines we are considering, i.e., the maximum number of kernels that can be concurrently executed at any time point (for our experiments, it is 2). In contrast, our approach maintains two dedicated streams for servicing memory operations, since we have two copy engines in the target platform and two compute streams (high and low priority) for servicing compute operations. Even if more than two kernels were waiting for dispatch, their concurrent execution yielded no benefits due to a lack of sufficient computing/memory resources and the overhead associated with maintaining more than two compute streams in the target device.

Observations: We present a comparative evaluation of the schedules produced by our proposed SMT-based approach and the baseline approach discussed in Section 1.

Memory Utilization: For each experiment E , we compute an average memory utilization measure for different scheduling schemes by recording the memory consumption over time in one hyper-period. This is done by selecting a window length δ and sub-dividing the hyper-period H into a total of $M = H/\delta$ intervals. For each interval x , we record the memory consumption mem_x . For a given scheduling scheme s and experiment E , the average memory utilization is computed as $mem_u(E, s) = \frac{\sum_x mem_x}{M}$. The value of δ varies for each experiment and is chosen as the minimum time between any two consecutive memory allocation/de-allocation events inside the hyper-period of an experiment. Any value of δ higher than this would miss accounting for all changes in memory consumption over time. We next define a gain measure that would help us evaluate the performance improvement in memory utilization between two scheduling schemes. This is depicted in the following equation. Given two scheduling schemes s_1 and s_2 , the percentage improvement in memory utilization of scheme s_2 over that of scheme s_1 for experiment E is computed as

$$\%Gain_{(s_1, s_2)}(E) = 100 - \frac{mem_u(E, s_1)}{mem_u(E, s_2)} * 100.$$

The percentage savings in memory utilization for our SMT-LS approach over the baseline ($\%Gain_{(SMT-LS, LSF-LS)}$) is calculated from the respective average memory utilization measures and are reported in column 5 of Table 1. One can observe that for each experiment, %-Gain is significantly high. This is because, for the baseline approach, weight tensors remain resident throughout the hyper-period as opposed to the SMT-assisted approach, where memory is allocated and de-allocated on the fly. In a similar manner, the percentage savings in memory utilization for our SMT-PMS-LS approach over that of baseline ($\%Gain_{(SMT-PMS-LS, LSF-LS)}$) is reported in column 6 of Table 1. The %-Gain values for SMT-PMS-LS are also significantly higher and are greater than that of SMT-LS. This is because the aggressive prefetching of weight tensors done as a result of the schedule produced by SMT-LS is reduced considerably by the PMS algorithm.

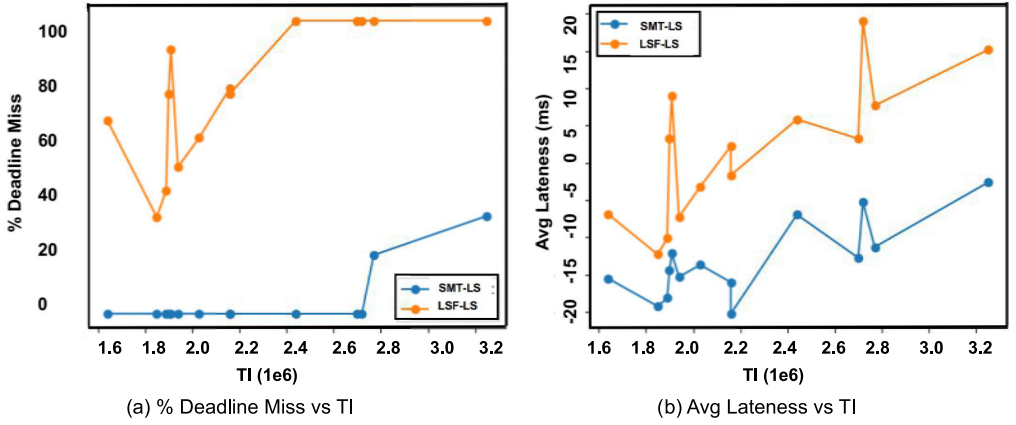


Fig. 12. Real-time scheduling performance.

Real-time Performance: For evaluating the real-time performance of the two scheduling schemes, we plot how the percentage of deadline misses and the average lateness values vary with increasing values of TI for each experiment in Figures 12(a) and 12(b), respectively. One can observe from Figure 12(a) that the SMT-assisted scheduling scheme clearly outperforms the baseline approach reporting deadline misses for a single job each for only two experiments *exp3* and *exp8*. In contrast, the baseline approach reports a range of deadline miss percentage values, varying from 37.5% to 100%. Also, note that in Figure 12(a), for experiment IDs 12 and 13, there exists a sudden spike in the deadline misses, even though the TI values are lesser than in subsequent experiments. This may be attributed to platform-level interference that occurs due to how close the jobs per task need to execute given their periodicity, as depicted in experiments 12 and 13 of Table 1 (in *italic*). Moreover, the high negative values of average lateness in Figure 12(b) indicates that in the SMT-assisted scheduling scheme, jobs finish considerably earlier than their respective deadlines. In comparison, the average lateness values become increasingly positive for the baseline approach, indicating that it cannot sustain higher throughput requirements. For the first three experiments with lower TI values, one can observe from Figure 12(b) that the average lateness value for the baseline approach is negative even though the deadline miss percentages in Figure 12(a) are greater than zero. This is because certain jobs scheduled by the baseline approach finish much earlier than their respective deadlines, contributing a higher negative lateness value to the average. Spikes similar to Figure 12(a) are also exhibited here, with much smaller (negative) peaks in the SMT approach. The greedy nature of the baseline approach in selecting jobs based on their slack values fails to enforce the required priorities for the jobs, causing some jobs to finish much earlier while causing others to end beyond their respective deadlines. In contrast, the SMT-assisted approach searches for the best possible mapping of operations to streams, allowing all jobs to finish within their deadlines.

PMS Experiments: Our next set of experiments aims to evaluate the efficacy of our proposed PMS algorithm in terms of savings with respect to memory utilization. This is done by considering all the experiment configurations used earlier and designing multiple experiments per configuration by suitably relaxing the deadlines of the tasks. For each configuration with two tasks, deadlines for task 1 were relaxed from 0 to 10 ms in steps of 2 ms, and deadlines of task 2 were relaxed from 0 to 15 ms in steps of 3 ms. Similarly, for each configuration with three tasks, the deadlines of each task are relaxed suitably. Thus, for each original experiment configuration, we get a total of 26 new configurations per the tasks' deadline relaxation parameters. This yields a total of 364 new

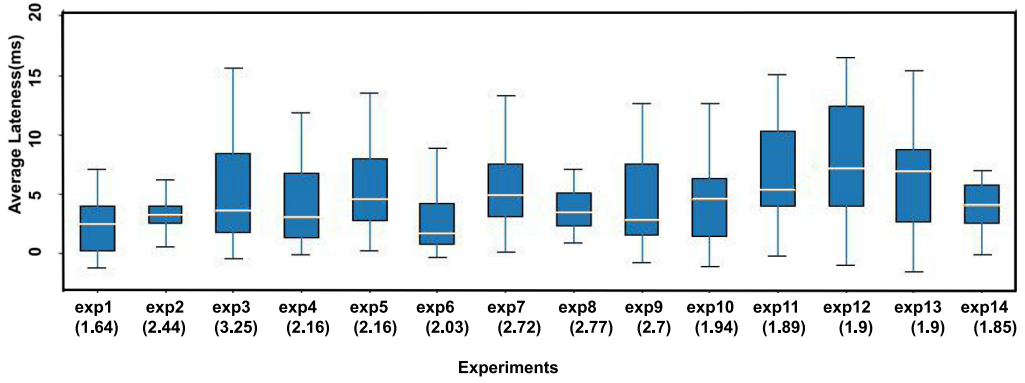


Fig. 13. Average Lateness observed after relaxation.

experiment configurations. Our goal is to evaluate the impact of deadline relaxation on average lateness and memory utilization for each of the schedules produced by the SMT-PMS-LS scheduler on each of the 364 new configurations.

We observe from the 364 schedules that as deadlines are relaxed progressively for both tasks, the PMS algorithm shifts jobs in the hyper-period, forcing them to finish beyond their original deadlines, thus increasing average lateness values. We illustrate the impact on average lateness with the help of suitable box plots in Figure 13. We consider the original 14 experiment configurations and capture the average lateness values in each of the 26 new configurations pertaining to an original experiment. Here, x-axis is marked with the original experiment IDs along with its throughput index value, and the y-axis denotes the average lateness values. Each individual box-plot for each of the 14 original experiments represents the average lateness distribution for each of the 26 new experiments obtained by relaxing values of both the tasks in the original experiment. Box-plot information represents five parameters—the minimum value, the first quartile (Q1), the median, the third quartile (Q3), and the maximum value of average lateness for each of the 26 new experiments. We observe that across all experiments, the average lateness values vary between -1.5 to 18 ms. Similarly, we obtain box plots of %Gain for memory utilization for baseline ($\%Gain_{(SMT-PMS-LS, LSF-LS)}$) for all the new experiment configurations and depict it in Figure 14. Here, the x-axis is labeled with original experiment IDs and their throughput index values, and the y-axis represents the %Gain in memory utilization. Like the average lateness box plot, each box plot provides the same five parameters for %Gain for every experiment. We observe across all experiments that percentage memory gain values vary between 78% to 98.5% . We observed a general trend of decrease in %Gain as the **throughput index (TI)** values for a configuration increased. This may be attributed to higher throughput index values correlating to higher compute requirements in less time. Consequently, there is little room for shifting memory/compute operations using our algorithms within a hyper-period and, subsequently lesser opportunity for savings in memory utilization. We next discuss experiment-specific observations concerning average lateness incurred and percentage savings in memory utilization.

- (1) The %Gain in memory utilization for experiments *exp1* and *exp14* is less when compared to that of the remaining experiments. The main reason for this particular behavior may be attributed to the task set's nature and the experiment's hyper-period configuration. As observed from Table 1, the jobs-per-task configurations for *exp1* and *exp14* are $[1, 2]$ and $[7, 1]$, respectively, indicating there is a skewed distribution of jobs with respect to the constituent tasks considered in the experiment. Furthermore, in experiments *exp1* and *exp14*, we have

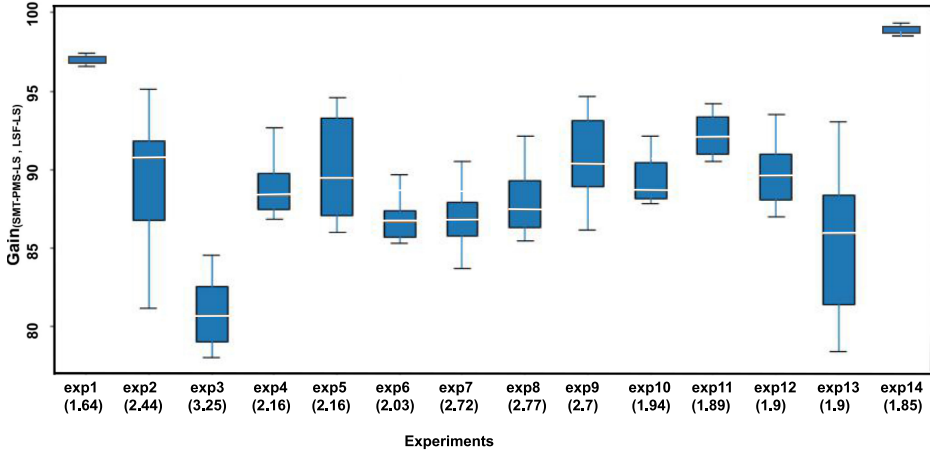


Fig. 14. Percentage improvements in gains after relaxation.

tasks τ_2 and τ_1 running with periods close to their running times, respectively. Thus, for τ_1 in *exp14*, we have seven jobs, each with absolute deadlines close to their running times. This implies that room for shifting is limited for such configurations, since free slots are constrained in size. A similar logic applies for τ_2 of *exp1*. We also observe that the range of average lateness values observed for *exp1* and *exp14* are relatively less when compared with the other experiments. This may be attributed to the same reason of there being relatively little room for shifting operations, as discussed earlier.

- (2) For experiment IDs *exp4*, *exp6*, *exp10*, *exp11* with the task configurations [3, 1], [4, 1], [5, 1], and [6, 1], respectively, deadline relaxation achieves a maximum of 6% in gain with respect to the PMS experiment without deadline extension. This may be attributed to the exact reason for the skewed distribution of jobs, as depicted in the earlier point. The %Gain for these experiments is relatively higher than *exp1* and *exp14*, since the periods considered for the tasks are more significant than their running times. Similarly, the range of average lateness values also does not differ much compared to the previous experiments.
- (3) An interesting observation may be noted in experiments *exp12* and *exp13*. Even though both configurations have the same (TI) value, the range of %Gain values observed are significantly more for *exp13* (maximum of 13% with respect to experiments with no deadline relaxation). This may be attributed to the fact that the tasks considered in *exp13* have relatively higher periods than their running times, allowing more free slots. For example, τ_1 has three jobs whose periods are twice that of their running times. Consequently, there is more room for shifting operations and reducing memory utilization. We observe similar behavior for *exp2* as well. Also, the average lateness values range is higher for those experiments where the range of %Gain values is higher.
- (4) We have conducted four experiments (*exp3*, *exp5*, *exp8*, and *exp9*) by considering three tasks with job configurations [1,1,1], [2,1,1], [2,1,2], [2,2,1], respectively. One can observe that the %Gain in memory utilization for *exp3* is less than that of the other three experiments. We note that *exp3* exhibits the highest throughput requirement among all 14 experiments and reports the least %Gain value. This is attributed to the fact that all three tasks arrive with the same period value of 30 ms. There is relatively less scope for the PMS algorithm to shift the operations even after the relaxation of deadlines.

From the above observations, it is evident that %Gain in terms of memory utilization obtained for an experiment after running our two-pass PMS algorithm is highly dependent upon (i) throughput index value, (ii) the distribution of jobs per task in a hyper-period, and (iii) the relative difference between task periods and task running times. We have observed that the general scope for reducing memory utilization decreases with higher throughput index values. Also, the difference between task periods and running times controls the number of available slots and, subsequently, the savings in memory utilization that can be obtained.

7 CONCLUSION AND FUTURE WORK

The proposed work provides a novel SMT-assisted scheduling approach capable of synthesizing operation-to-resource mapping decisions for co-scheduling multiple inference pipelines on resource-constrained edge devices. We also envisioned a multi-pass memory optimization scheme that processes the schedules generated by the SMT optimizer and modifies the mapping decisions accordingly to minimize the memory consumption per unit time during the execution of the pipelines. Our experiments indicate that the proposed methodologies outperform the state-of-the-art techniques in terms of memory utilization while minimizing the worst-case response times. Future endeavors include investigating real-time scheduling strategies while optimizing for memory consumption in an online problem setting. In this context, we plan to leverage learning-based mechanisms for searching through the large state space of scheduling decisions for the same. An immediate future extension of this work could be implementing inference pipelines for a few more ADAS tasks and evaluating their performance on the SMT-based schedule and PMS approach. Another future goal to focus on is identifying strategies that efficiently explore the scheduling state space for online real-time scheduling.

REFERENCES

- [1] Tanya Amert, Nathan Otterness, Ming Yang, James H. Anderson, and F. Donelson Smith. 2017. GPU scheduling on the NVIDIA TX2: Hidden details revealed. In *IEEE Real-Time Systems Symposium (RTSS'17)*. IEEE, 104–115.
- [2] Clark Barrett and Cesare Tinelli. 2018. *Satisfiability Modulo Theories*. Springer International Publishing, Cham, 305–343.
- [3] Sanjoy Baruah, Bipasa Chattopadhyay, Haoan Li, and Insik Shin. 2014. Mixed-criticality scheduling on multiprocessors. *Real-time Syst.* 50, 1 (2014), 142–177.
- [4] Soroush Bateni, Zhendong Wang, Yuankun Zhu, Yang Hu, and Cong Liu. 2020. Co-optimizing performance and memory footprint via integrated CPU/GPU memory management, an implementation on autonomous driving platform. In *Real-Time and Embedded Technology and Applications Symposium (RTAS'20)*. IEEE, 310–323.
- [5] Nicola Capodici, Roberto Cavicchioli, Marko Bertogna, and Aingara Paramakuru. 2018. Deadline-based scheduling for GPU with preemption support. In *IEEE Real-Time Systems Symposium (RTSS'18)*. IEEE, 119–130.
- [6] Samarjit Chakraborty, Mohammad Abdullah Al Faruque, Wanli Chang, Dip Goswami, Marilyn Wolf, and Qi Zhu. 2016. Automotive cyber-physical systems: A tutorial introduction. *IEEE Design Test* 33, 4 (2016), 92–108.
- [7] Tiffany Yu-Han Chen, Lenin Ravindranath, Shuo Deng, Paramvir Bahl, and Hari Balakrishnan. 2015. Glimpse: Continuous, real-time object recognition on mobile devices. In *13th ACM Conference on Embedded Networked Sensor Systems*. ACM, 155–168.
- [8] Xiaoming Chen, Danny Ziyi Chen, Yinhe Han, and Xiaobo Sharon Hu. 2018. moDNN: Memory optimal deep neural network training on graphics processing units. *IEEE Trans. Parallel Distrib. Syst.* 30, 3 (2018), 646–661.
- [9] Intel corporation. 2020. OpenVINO. Retrieved from <https://docs.openvino toolkit.org/>.
- [10] Abdul Dakkak, Cheng Li, Simon Garcia De Gonzalo, Jinjun Xiong, and Wen-mei Hwu. 2019. TRIMS: Transparent and isolated model sharing for low latency deep learning inference in function-as-a-service. In *12th International Conference on Cloud Computing (CLOUD'19)*. IEEE, 372–382.
- [11] Madhuchhanda Dasgupta, Oishila Bandyopadhyay, and Sanjay Chatterji. 2019. Automated helmet detection for multiple motorcycle riders using CNN. In *Conference on Information and Communication Technology*. IEEE, 1–4.
- [12] Wen-mei W. Hwu and David B. Kirk. 2016. *Programming Massively Parallel Processors* (3rd ed.). Morgan Kaufmann.
- [13] Shaoxia Fang, Lu Tian, Junbin Wang, Shuang Liang, Dongliang Xie, Zhongmin Chen, Lingzhi Sui, Qian Yu, Xiaoming Sun, Yi Shan, and Yu Wang. 2018. Real-time object detection and semantic segmentation hardware system with deep learning networks. In *International Conference on Field-Programmable Technology (FPT'18)*. IEEE, 389–392.

- [14] Anirban Ghose, Srijeeta Maity, Arijit Kar, and Soumyajit Dey. 2021. Orchestration of perception systems for reliable performance in heterogeneous platforms. In *Design, Automation & Test in Europe Conference & Exhibition (DATE'21)*. IEEE, 1757–1762.
- [15] Juan Gómez-Luna, Izzat El Hajj, Li-Wen Chang, Victor García-Floreszx, Simon Garcia De Gonzalo, Thomas B. Jablin, Antonio J. Pena, and Wen-mei Hwu. 2017. Chai: Collaborative heterogeneous applications for integrated-architectures. In *International Symposium on Performance Analysis of Systems and Software (ISPASS'17)*. IEEE, 43–54.
- [16] Xiaohong Han, Jun Chang, and Kaiyuan Wang. 2021. Real-time object detection based on YOLO-v2 for tiny vehicle object. *Proced. Comput. Sci.* 183 (2021), 61–72.
- [17] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. DOI: <https://doi.org/10.48550/ARXIV.1704.04861>
- [18] Rachel Huang, Jonathan Pedoeem, and Cuixian Chen. 2018. YOLO-LITE: A real-time object detection algorithm optimized for non-GPU computers. In *IEEE International Conference on Big Data (Big Data'18)*. IEEE, 2503–2510.
- [19] Loc N. Huynh, Youngki Lee, and Rajesh Krishna Balan. 2017. DeepMon: Mobile GPU-based deep learning framework for continuous vision applications. In *15th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 82–95.
- [20] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level Accuracy with 50x Fewer Parameters and < 0.5MB Model Size. DOI: <https://doi.org/10.48550/ARXIV.1602.07360>
- [21] Cheng Ji, Fan Wu, Zongwei Zhu, Li-Pin Chang, Huanghe Liu, and Wenjie Zhai. 2021. Memory-efficient deep learning inference with incremental weight loading and data layout reorganization on edge systems. *J. Syst. Archit.* 118 (2021), 102183. DOI: <https://doi.org/10.1016/j.sysarc.2021.102183>
- [22] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. 2018. Chameleon: Scalable adaptation of video analytics. In *Conference of the ACM Special Interest Group on Data Communication*. ACM, 253–266.
- [23] Ziheng Jiang, Tianqi Chen, and Mu Li. 2018. *Efficient Deep Learning Inference on Edge Devices*. ACM SysML, Stanford, CA.
- [24] Woosung Kang, Kilho Lee, Jinkyu Lee, Insik Shin, and Hoon Sung Chwa. 2021. LaLaRAND: Flexible layer-by-layer CPU/GPU scheduling for real-time DNN tasks. In *Real-Time Systems Symposium (RTSS'21)*. IEEE, 329–341.
- [25] Kye-Hyeon Kim, Sanghoon Hong, Byungseok Roh, Yeongjae Cheon, and Minje Park. 2016. PVANET: Deep but Lightweight Neural Networks for Real-time Object Detection. (2016). DOI: <https://doi.org/10.48550/ARXIV.1608.08021>
- [26] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (2017), 84–90.
- [27] Chao Li, Yi Yang, Min Feng, Srimat Chakradhar, and Huiyang Zhou. 2016. Optimizing memory efficiency for deep convolutional neural networks on GPUs. In *International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 633–644.
- [28] Jinyang Li, Runyu Ma, Vikram Sharma Mailthody, Colin Samplawski, Benjamin Marlin, Songqing Chen, Shuochao Yao, and Tarek Abdelzaher. 2021. Towards an accurate latency model for convolutional neural network layers on GPUs. In *Military Communications Conference (MILCOM'21)*. IEEE, 904–909. DOI: <https://doi.org/10.1109/MILCOM52596.2021.9652907>
- [29] Srijeeta Maity, Anirban Ghose, Soumyajit Dey, and Swarnendu Biswas. 2020. Thermal load-aware adaptive scheduling for heterogeneous platforms. In *33rd International Conference on VLSI Design and 19th International Conference on Embedded Systems (VLSID'20)*. IEEE, 125–130.
- [30] Srijeeta Maity, Anirban Ghose, Soumyajit Dey, and Swarnendu Biswas. 2021. Thermal-aware adaptive platform management for heterogeneous embedded systems. *ACM Trans. Embed. Comput. Syst.* 20, 5s (2021), 1–28.
- [31] Abhinandan Majumdar, Leonardo Piga, Indrani Paul, Joseph L. Greathouse, Wei Huang, and David H. Albonesi. 2017. Dynamic GPGPU power management using adaptive model predictive control. In *International Symposium on High Performance Computer Architecture (HPCA'17)*. IEEE, 613–624.
- [32] Arezou Mohammadi and Selim G. Akl. 2005. *Scheduling Algorithms for Real-time Systems*. Technical Report. School of Computing Queens University. Citeseer.
- [33] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [34] Nvidia. 2020. CUDA C++ Programming Guide. Retrieved from <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [35] Nvidia. 2020. NVIDIA Drive Perception. Retrieved from <https://developer.nvidia.com/drive/drive-perception>.
- [36] Ignacio Sañudo Olmedo, Nicola Capodici, Jorge Luis Martinez, Andrea Marongiu, and Marko Bertogna. 2020. Dissecting the CUDA scheduling hierarchy: A performance and predictability perspective. In *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'20)*. IEEE, 213–225.

- [37] Qiushi Pan, Yutong Guo, and Zhiliang Wang. 2019. A scene classification algorithm of visual robot based on Tiny YOLO v2. In *Chinese Control Conference (CCC'19)*. IEEE, 8544–8549.
- [38] Yury Pisarchyk and Juhyun Lee. 2020. Efficient Memory Management for Deep Neural Net Inference. DOI:<https://doi.org/10.48550/ARXIV.2001.03288>
- [39] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [40] Franz Rammig, Michael Ditze, Peter Janacik, Tales Heimfarth, Timo Kerstan, Simon Oberthuer, and Katharina Stahl. 2009. Basic concepts of real time operating systems. In *Hardware-dependent Software*. Springer, Dordrecht, 15–45.
- [41] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 779–788.
- [42] Joseph Redmon and Ali Farhadi. 2017. YOLO9000: Better, faster, stronger. In *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 7263–7271.
- [43] Joseph Redmon and Ali Farhadi. 2018. YOLOv3: An Incremental Improvement. DOI:<https://doi.org/10.48550/ARXIV.1804.02767>
- [44] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE/ACM, 1–13.
- [45] Karattupalayam Chidambaram Saranya, Arunkumar Thangavelu, Ashwin Chidambaram, Sharan Arumugam, and Sushant Govindraj. 2020. Cyclist detection using Tiny YOLO v2. In *Soft Computing for Problem Solving*. Springer, 969–979.
- [46] Ulzhalgas Seidaliyeva, Manal Alduraibi, Lyazzat Ilibayeva, and Akhan Almagambetov. 2020. Detection of loaded and unloaded UAV using deep neural network. In *4th International Conference on Robotic Computing (IRC'20)*. IEEE, 490–494.
- [47] S. B. Shriram, Anshuj Garg, and Purushottam Kulkarni. 2019. Dynamic memory management for GPU-based training of deep neural networks. In *International Parallel and Distributed Processing Symposium (IPDPS'19)*. IEEE, 200–209.
- [48] Tensorflow. 2020. TensorFlow-lite. Retrieved from <https://www.tensorflow.org/lite>.
- [49] Mohamed Wahib, Haoyu Zhang, Truong Thao Nguyen, Aleksandr Drozd, Jens Domke, Lingqi Zhang, Ryousei Takano, and Satoshi Matsuoka. 2020. Scaling distributed deep learning workloads beyond the memory capacity with KARMA. In *International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [50] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. SuperNeurons: Dynamic GPU memory management for training deep neural networks. In *23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, 41–53.
- [51] Jian Wei, Jianhua He, Yi Zhou, Kai Chen, Zuoyin Tang, and Zhiliang Xiong. 2019. Enhanced object detection with deep convolutional neural networks for advanced driving assistance. *IEEE Trans. Intell. Transport. Syst.* 21, 4 (2019), 1572–1583.
- [52] Yecheng Xiang and Hyoseung Kim. 2019. Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference. In *IEEE Real-Time Systems Symposium (RTSS'19)*. IEEE, 392–405.
- [53] Ming Yang, Shige Wang, Joshua Bakita, Thanh Vu, F. Donelson Smith, James H. Anderson, and Jan-Michael Frahm. 2019. Re-thinking CNN frameworks for time-sensitive autonomous-driving applications: Addressing an industrial challenge. In *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'19)*. IEEE, 305–317.
- [54] Shuochao Yao, Shaohan Hu, Yiran Zhao, Aston Zhang, and Tarek Abdelzaher. 2017. DeepSense: A unified deep learning framework for time-series mobile sensing data processing. In *26th International Conference on World Wide Web*. ACM, 351–360.
- [55] Pavly Salah Zaki, Marco Magdy William, Bolis Karam Soliman, Kerolos Gamal Alexsan, Kerolos Khalil, and Magdy El-Moursy. 2020. Traffic Signs Detection and Recognition System using Deep Learning. DOI:<https://doi.org/10.48550/ARXIV.2003.03256>
- [56] Yuanyuan Zheng and Jun Ge. 2021. Binocular intelligent following robot based on YOLO-LITE. In *MATEC Web of Conferences*, Vol. 336. EDP Sciences, 03002.
- [57] Husheng Zhou, Soroush Batani, and Cong Liu. 2018. S3DNN: Supervised streaming and scheduling for GPU-accelerated real-time DNN workloads. In *Real-Time and Embedded Technology and Applications Symposium (RTAS'18)*. IEEE, 190–201.

Received 10 July 2022; revised 26 November 2022; accepted 4 December 2022